



M362 Unit 6

UNDERGRADUATE COMPUTING

Developing concurrent distributed systems



The database tier:
Handling transactions
and concurrency

Unit **6**

This publication forms part of an Open University course M362 *Developing concurrent distributed systems*. Details of this and other Open University courses can be obtained from the Student Registration and Enquiry Service, The Open University, PO Box 197, Milton Keynes MK7 6BJ, United Kingdom: tel. +44 (0)845 300 60 90, email general-enquiries@open.ac.uk

Alternatively, you may visit the Open University website at <http://www.open.ac.uk> where you can learn more about the wide range of courses and packs offered at all levels by The Open University.

To purchase a selection of Open University course materials visit <http://www.ouw.co.uk>, or contact Open University Worldwide, Michael Young Building, Walton Hall, Milton Keynes MK7 6AA, United Kingdom for a brochure. tel. +44 (0)1908 858793; fax +44 (0)1908 858787; email ouw-customer-services@open.ac.uk

The Open University
Walton Hall
Milton Keynes
MK7 6AA

First published 2008.

Copyright © 2008 The Open University.

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, transmitted or utilised in any form or by any means, electronic, mechanical, photocopying, recording or otherwise, without written permission from the publisher or a licence from the Copyright Licensing Agency Ltd. Details of such licences (for reprographic reproduction) may be obtained from the Copyright Licensing Agency Ltd, Saffron House, 6–10 Kirby Street, London EC1N 8TS; website <http://www.cla.co.uk>

Open University course materials may also be made available in electronic formats for use by students of the University. All rights, including copyright and related rights and database rights, in electronic course materials and their contents are owned by or licensed to The Open University, or otherwise used by The Open University as permitted by applicable law.

In using electronic course materials and their contents you agree that your use will be solely for the purposes of following an Open University course of study or otherwise as licensed by The Open University or its assigns.

Except as permitted above you undertake not to copy, store in any medium (including electronic storage or use in a website), distribute, transmit or retransmit, broadcast, modify or show in public such electronic materials in whole or in part without the prior written consent of The Open University or in accordance with the Copyright, Designs and Patents Act 1988.

Edited and designed by The Open University.

Typeset by SR Nova Pvt. Ltd, Bangalore, India.

Printed and bound in the United Kingdom by Martins the Printers, Berwick-upon-Tweed.

ISBN 978 0 7492 1594 1

1.1

1	Introduction	6
1.1	The aims of this unit	6
2	Databases – some background	7
3	Transactions	8
3.1	Atomicity and transactions	8
3.2	ACID properties	9
3.3	Transfer example	10
3.4	Transaction states	11
3.5	Transaction processing systems	12
4	Theory of serialisability	15
4.1	Schedules	15
4.2	Serialisability	18
4.3	Conflicting operations	19
4.4	Testing for serialisability	20
5	Concurrency control for transactions	28
5.1	Two-phase locking	28
5.2	Time-stamp ordering	32
5.3	Optimistic concurrency control	37
5.4	Comparison of methods for concurrency control	37
6	Crash resilience and data recovery	39
6.1	Transactions: failure atomicity and durability	39
6.2	Persistence	40
6.3	Recovery	40
7	Communication: JDBC	43
7.1	DBMS functionality and the relational model	43
7.2	Setting up a connection	44
7.3	DBMS transaction support	46
7.4	The <code>Connection</code> interface and SQL	47
7.5	JDBC – transaction isolation levels	49

8	Distributed databases	53
8.1	Fragmentation and transparency	53
8.2	Data replication	54
8.3	Distributed transactions	56
8.4	Relationships between problems	57
9	Summary	60
	References	62
	Glossary	63
	Index	69



M362 COURSE TEAM

Affiliated to The Open University unless otherwise stated.

Chair, author and academic editor

Janet van der Linden

Authors

Anton Dil

Brendan Quinn

Michel Wermelinger

Critical readers and testers

Henryk Krajinski

Barbara Segal

Mark Thomas

Richard Walker

Yijun Yu

External assessor

Aad van Moorsel, Newcastle University

Software development

Ivan Dunn, Consultant

Course management

Linda Landsberg

Carrie Lewis

Barbara Poniatowska

Julia White

Media development staff

Ian Blackham, Editor

Sarah Gamman, Contracts Executive

Jennifer Harding, Editor

Phillip Howe, Media Assistant

Martin Keeling, Media Assistant

Callum Lester, Software Developer

Andy Seddon, Media Project Manager

Sue Stavert, Technical Testing Team

Andrew Whitehead, Designer and Graphic Artist

Thanks are due to the Desktop Publishing Unit of the Faculty of Mathematics, Computing and Technology.

1

Introduction

In the previous unit we described a multi-tiered architecture for distributed applications. In this unit we study the lowest level of this architecture where we deal with the storage of data.

This tier is very important because data is a valuable resource for an organisation. For example, this is the tier that is used to store all the details of the customer bank accounts in a bank, the medical records of a hospital's patients, or all the invoices for instruments ordered by customers of a music shop. In most computer systems a database is used to store such data, so in this unit we will give a brief overview of databases.

However, the main emphasis in this unit is not so much on the actual storage of data, but rather the mechanisms to keep data up to date in a consistent way. We will continue our study of concurrency, but now at a higher level than in Block 1. We are here dealing with systems that are inherently concurrent, because they have to react to events that happen outside the computer system. Several such events may happen together, such as a number of users wanting to access data objects simultaneously. We will study some of the algorithms that have been developed in order to control tightly access to data, generally known as concurrency control algorithms.

Another issue is that of a crash happening in the middle of an important operation. This can be a problem for centralised systems, but in distributed systems there are even more things that can go wrong. We will study some mechanisms that help systems to recover from such crashes, so that when the systems start up again the data can be put in an orderly state.

We will also study the communication mechanism between a Java application and a database, and the final section takes a look at some system design issues for distributed databases.

1.1 The aims of this unit

In this unit you will study:

- ▶ what a database is and how it is used (Section 2);
- ▶ how we can safely change or view the data in a database, and how this is done through so-called transactions (Sections 3, 4 and 5);
- ▶ different mechanisms to aid systems to recover from crashes (Section 6);
- ▶ how to set up the communication between a database and a Java application (Section 7);
- ▶ distributed databases (Section 8).

This unit mainly involves reading, together with exercises, apart from Section 7 which has some practical activities on how to program the communication between a database and a Java application.

2

Databases – some background

In this section we outline briefly what a database is and what it aims to achieve, and we explore the differences between the way data is stored on a database and the way it is held in memory.

Database

Databases have been around for a long time, either as paper-based information systems, or, since the 1960s, as computerised systems. A **database** is an organised collection of data. However, when we talk about computerised systems, we generally understand a database to be a collection of records stored in a systematic way, so that a computer program can consult it to answer queries or to update the data. A database is typically used when we are dealing with large amounts of data that need to be stored independent of any running program.

A typical user of a database system will want to make queries relating to the data. Taking the scenario of a music shop, examples of such queries could be as follows: 'How many violins are in stock?' or 'Show all trumpets below a certain price'.

DBMS

The owners of the database are concerned with keeping the data in the database as accurate and up to date as possible. The computer program used to manage and query a database is known as a **database management system (DBMS)**. The main responsibility of the DBMS is to maintain the interaction with the user, and to organise the writing and reading of data. Note that such a DBMS must be a concurrent system, because it is required to be able to handle a number of interacting clients simultaneously.

Persistent storage

In *Unit 2* we discussed different forms of memory: volatile and persistent. Data that is held in volatile memory will disappear when the device is switched off. In contrast, data that is stored in a database is said to be persistent, which means that the data outlives the execution of the software program that created it. We refer to a database as a form of **persistent storage**. If a program stops and restarts, data that was stored in a database can be reloaded and, provided that no other program has changed the data in the meantime, it will be the same as it was in the previous invocation of the program.

This course is about neither database design nor the use of database query languages. Our interest in databases stems from their role in distributed systems at the bottom tier of multi-tiered systems: providing a persistent data storage mechanism.

Another way of making data persistent is to store the data in a file. You saw an example of this in Activity 1.1 of *Unit 1*, with the use of a comma-separated values (CSV) file.



Courses and qualifications. Consult the Open University website for details about database courses.

3 Transactions

In *Unit 3* we studied the concept of atomicity, and we distinguished between fine-grained and coarse-grained actions. In particular, we said that a coarse-grained action is atomic if it gives the appearance of being indivisible, even though it consists of more than one machine instruction. In this section we discuss the concept of a **transaction**, which is a sequence of atomic actions, where we want the whole sequence to be performed in its entirety, or not performed at all.

The classic example of a transaction is a transfer of funds in a bank application. Assume that we have two bank accounts, denoted by X and Y , and that we need to transfer £100 from account X to account Y . This would consist of the following sequence of operations.

```
transfer =  
  1 X.read()  
  2 X = X - 100  
  3 X.write()  
  4 Y.read()  
  5 Y = Y + 100  
  6 Y.write()
```

It is important that either all the operations take place, or none of the operations takes place. If only the first three take place, for example, without steps 4 to 6, this would lead to data consistency problems. That is, it would appear that money has been lost, and thus the database has been put into an inconsistent state. For a transfer transaction the consistency requirement is that the sum of X and Y remains unchanged by the execution of the transaction.

The notion of a transaction was developed in the context of databases and refers to a number of database operations that together should be treated as one. Particularly in banking systems, transactions enable operations such as transferring money from one account to another to be made secure and consistent.

Transactions and the management of transactions is a crucial part of enterprise application development. Most of the large enterprise applications in the domains of finance and online shopping rely heavily on transaction processing for delivering their business functionality.

In this section we will define what is meant by a transaction and discuss its properties. We then briefly look at the software systems that help to process transactions.

3.1 Atomicity and transactions

A transaction seeks to address the issue of atomicity at an even higher level than was discussed in *Unit 3*. We mentioned the example of a money transfer in a banking system. The *transfer* transaction consists of a number of smaller steps. The important point here is that by defining *transfer* as a transaction, the component operations now belong together, and one cannot be separated from the others. The transfer operation is atomic.

Transactions also differ from the atomic operations we saw in *Unit 3* in having an additional quality. For a transaction the following should be true:

- ▶ if it terminates normally, all its effects are made permanent;
- ▶ if it fails, it is as if nothing had happened.

So as well as giving the appearance of indivisibility, an idea familiar from *Unit 3*, transactions also exhibit an all-or-nothing atomicity which is guaranteed if there is some kind of failure. This property is known as **failure atomicity**.

Transactions can fail for a number of different reasons. There may be an error in the input data, or there may be some other problem related to the nature of the transaction itself, which means that the transaction cannot continue. If we are dealing with transactions relating to bank accounts, it may be that a wrong account number was entered. It is also possible that an account did not have a high enough balance, so that a transfer from the account had to be forbidden. In such a case we say that the transaction failed and that it had to be **aborted**.

Another cause of transaction failure can be a crash in part of the system. It may be that the client application or the server comes to a halt. There may also be network problems leading to communication failure. System failure is discussed in Section 6.

When a transaction has been carried out to completeness without encountering any problems, it can be committed. This means that its results are made permanent, and can be shown to other transactions. This is the binding nature of the transaction and refers back to the fact that transactions have their origin in contract law, as explained by Gray.

In making a contract, two or more parties negotiate for a while and then make a deal. The deal is made binding by the joint signature of a document or by some other act (as simple as a handshake or nod). If the parties are rather suspicious of one another or just want to be safe, they appoint an intermediary ... to coordinate the commitment of the transaction.

(Gray, 1981)

If a transaction is committed, all parties can rely on that transaction having taken place. The 'commit' moment is therefore sometimes described as a 'no turning back' moment.

3.2 ACID properties

Putting all the above points together, we can say that a transaction is required to have a number of *desirable properties*. Throughout this unit you will learn that a lot of effort goes into ensuring that these properties are achieved. The properties are generally referred to as the **ACID properties** and they are defined as follows.

- ▶ Atomicity – either all the operations of a transaction are performed, or none at all.
- ▶ Consistency – a transaction transforms the system from one consistent state to another.
- ▶ Isolation – a transaction cannot reveal any part of its state to other transactions until the transaction is committed.
- ▶ Durability – once a transaction is committed, the system must guarantee that the result will persist, even if there are subsequent system failures.

The atomicity property specifies that a transaction should not be allowed to do only part of its job, because that could lead to an inconsistent state. We already saw that if we complete only the first half of the *transfer* transaction, then money effectively disappears from the bank system.

The ACID acronym is based on the first letter of each property.

By consistency we mean that the rules that specify the state of the database are not infringed and that there are no contradictory results. We gave the example of the sum of two accounts remaining the same in the case of a transfer from one account to another. It is the responsibility of the application developer to ensure that individual transactions are coded correctly, and that each transaction leaves the database in a consistent state.

However, it is possible that during the execution of the transaction the database is temporarily inconsistent. This is not a problem, as long as no other transaction is allowed to look at that temporary inconsistent state. That is what the isolation property specifies: transactions must be unaware of other concurrently executing transactions and should not be allowed to see the intermediate results of other transactions halfway through their execution.

The isolation property is the only property that may sometimes be relaxed; however, we shall see later that such relaxation may cause problems that then have to be addressed.

3.3 Transfer example

A transaction is usually initiated by a user application, and can be written in a number of different languages, for example SQL, Java or C++. Here we will look at the example of the transfer transaction that we met earlier, and demonstrate how this might be coded in Java, using the **Java Transaction API (JTA)**. JTA provides a standard interface to demarcate transactions.

Rather than the `read` and `write` statements, we will use a more realistic example involving objects of type `BankAccount` with the following methods.

Table 1 Operations for class `BankAccount`

Operation	Description
<code>credit(amount)</code>	Adds the amount to the balance stored by the object.
<code>debit(amount)</code>	Takes away the amount from the balance.
<code>getBalance()</code>	Reads the current balance and returns its value.
<code>addInterest()</code>	Calculates the interest and adds it to the balance of the account.

In the following code sample we assume we have access to an object of type `UserTransaction` (part of the JTA specification) referenced by `transaction`. The sample uses two `BankAccount` objects, a `customerAcc` object and a `shopAcc` object, and the aim is to transfer £250 from the customer's account to the shop's account.

```

transaction.begin(); //1
if (customerAcc.getBalance() >= 250) //2
{
    customerAcc.debit(250); //3
    shopAcc.credit(250); //4
    transaction.commit(); //5
}
else //6
{
    System.out.println("Insufficient funds for transfer"); //7
    transaction.rollback(); //8
}

```

In Section 7 we will briefly return to the issue of the JTA specification and when to use it.

The start of the transaction is indicated by the `transaction.begin` statement. This is then followed by a check to see whether the object (the customer's account here) is in a legal state so that the transaction is allowed to go ahead. In this example, a legal state means that there are sufficient funds in the account for the `debit` to go ahead. If it is not in a legal state, the transaction will abort with the `transaction.rollback` statement. A successful initial state will mean that the two operations for `debit` and `credit` will be invoked, followed by `transaction.commit` to indicate the successful completion of the transaction.

Note that there are two ways to indicate the end of the transaction: `commit` and `rollback`.

3.4 Transaction states

In the above subsections we have mentioned a number of reasons why a transaction may not complete its execution successfully. It is important to be clear about what we mean by a successful completion of a transaction, which is why we establish a simple abstract transaction model. The model makes a conceptual distinction between the different states that a transaction may be in.

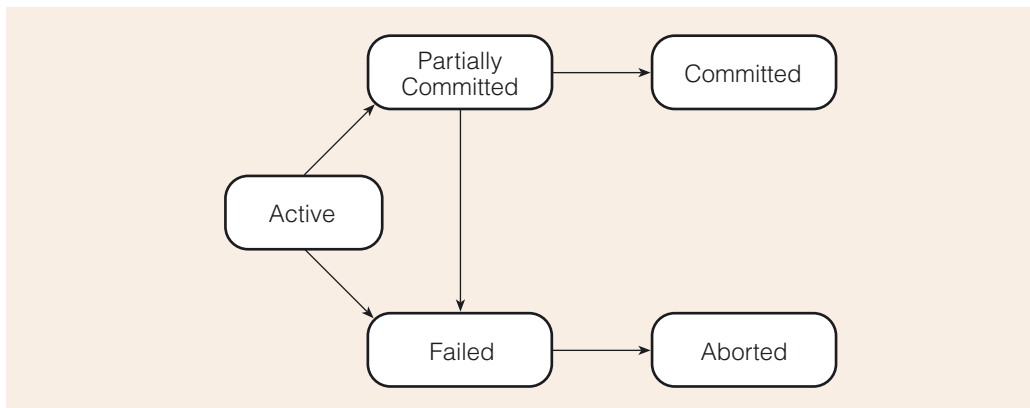


Figure 1 State diagram of a transaction

The transaction model in Figure 1 shows five different states.

- ▶ **Active** – the initial state a transaction is in when it is first started. While the transaction is executing it remains in this active state.
- ▶ **Partially Committed** – the state after the final operation has been executed.
- ▶ **Committed** – the state after successfully completing the transaction. All the changes to the database made by the transaction will persist even if there is a system failure.
- ▶ **Failed** – the state when the discovery is made that normal execution cannot proceed.
- ▶ **Aborted** – the state when the transaction has been rolled back, that is, all its effects are undone, and the database is restored to the state it was in prior to the start of the transaction.

When a transaction is started it is given a unique transaction ID number, and it is in the Active state. At the moment it has completed its final operation, it will enter into the Partially Committed state. It is possible that all the output from the transaction is still temporarily held in main memory (volatile memory), and that if a system crash were to occur at that precise moment, the updates would not have been made permanent. The transaction remains in the Partially Committed state while the system starts writing information to disk. Only when the last bit of information is written to disk, can the transaction enter the Committed state.

A transaction may fail for a number of reasons: there may be logical errors (e.g. an insufficient balance or the wrong account number) or there may be hardware or software errors unrelated to the internal logic of the transaction. A transaction that fails must be rolled back, and will enter the Aborted state. Depending on the reason for the failure, the system can then do one of the following.

- ▶ Restart the transaction. This takes place if the failure was caused by some hardware or software failure. The restarted transaction is treated as a new transaction and receives a new ID.
- ▶ Kill the transaction. This takes place in the case of internal logical errors in the transaction or because the input was bad. It may be that the problem that caused the failure can be corrected only by rewriting the application program.

Exercise 1

Look at the code for the `UserTransaction` in Subsection 3.3 to transfer money from a customer's account to a shop's account, using the JTA interface. The code steps have been indicated with numbered comments. Using the abstract transaction state model in Figure 1, indicate for each code statement which state the transaction is in after it has completed the statement contained in that line of code.

Discussion

```

transaction.begin(); //1 – Active
if (customerAcc.getBalance() >= 250) //2 – Active
{
    customerAcc.debit(250); //3 – Active
    shopAcc.credit(250); //4 – Partial commit
    transaction.commit(); //5 – Committed
}
else //6 – Failed
{
    System.out.println("Insufficient funds for transfer"); //7 – Failed
    transaction.rollback(); //8 – Aborted
}

```

3.5 Transaction processing systems

In large systems where a large number of clients access a database to update or query data, the load in terms of numbers of transactions needing processing could be very heavy. We have already indicated that for distributed systems responsiveness is an important factor, and it would be deemed unsatisfactory if transactions had to be carried out one at a time, with no new transaction able to begin until the current one had committed. If no concurrency was allowed, any particular transaction might be faced with a long wait until all the earlier transactions had completed. Furthermore, the throughput of such a system would be drastically reduced, because, if for some reason one transaction could not make progress, it would not be possible for other transactions to run instead. To get satisfactory performance it is essential for transactions to execute concurrently, allowing each transaction to do useful work whenever the opportunity is presented.

However, allowing concurrency raises a serious problem. How can we ensure that transactions don't interfere with one another and put the database information into an inconsistent state?

It is the task of a **transaction processing system** (also known as a TP system or a TP monitor) to manage the correct and efficient execution of transactions. TP systems, which date back to the early 1960s, were the first form of middleware. The main goal of TP systems is to support the efficient sharing of resources among as many concurrent customers as possible. TP systems are located between the clients and the database, as shown in Figure 2, which also shows the major components that make up a TP system.

One of the earliest TP monitors was IBM's customer information and control system (CICS) which was developed in the late 1960s, and is still in use today.

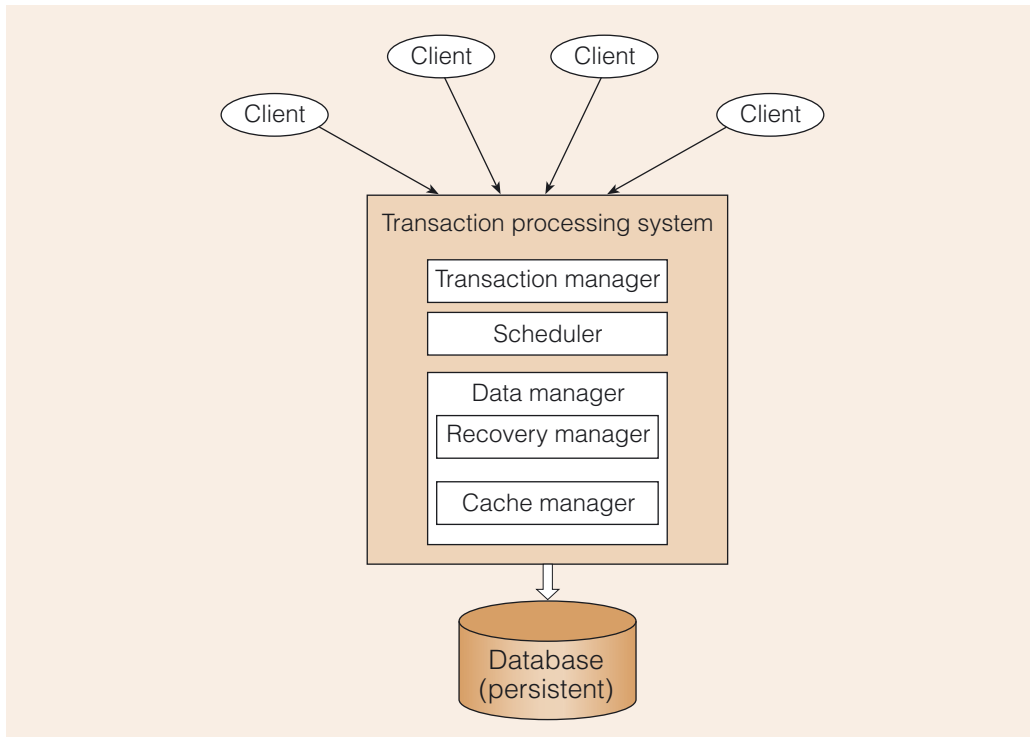


Figure 2 The software architecture of a transaction processing system

Each component within the transaction processing system is responsible for a particular aspect of the processing of transactions:

- ▶ a **transaction manager** provides an interface between the clients and the system, validates requests and passes operations to the scheduler;
- ▶ a **scheduler** puts together a schedule for all the component operations, making sure that all the operations are executed in the right order so that transactions do not interfere with each other;
- ▶ a **recovery manager** deals with the recovery from crashes;
- ▶ a **cache manager** organises the objects in main memory and interfaces to persistent memory (database).

In the following sections we will learn how all the parts of the TP system work together to enable the concurrent execution of transactions, while making sure that the ACID properties will hold.

SAQ 1

- (a) Compare the concept of a transaction with that of a coarse-grained atomic action. What are their differences and similarities?
- (b) What are the desirable properties for transactions?
- (c) What are the transaction states in the transaction state model?

- (d) When a transaction is in the Partially Committed state, it has not fully completed. What are the possible next states? Describe the circumstances under which it would progress to each of them.
- (e) What is the role of a TP system?

ANSWER.....

- (a) A transaction and a coarse-grained atomic action both consist of several operations, but these are treated as belonging together, and hence both are atomic. They both give the appearance of indivisibility.
- The difference is that a transaction also has the property of failure atomicity:
- ▶ if there is a failure part-way through the execution of the transaction, all the partly done operations are undone, so that it is as if nothing has happened;
 - ▶ if the transaction completes, the results of the transaction are made durable (stored in persistent storage).
- (b) There are four desirable (ACID) properties: atomicity, consistency, isolation and durability.
- (c) The transaction model specifies the following transaction states: Active, Failed, Partially Committed, Committed and Aborted.
- (d) When a transaction is Partially Committed it can progress to the Committed state in the case where all the data involved in the transaction is written to disk and made durable. On the other hand, if there is a problem in making the data durable, the transaction is said to have Failed. The effects of the transaction are then undone, i.e. rolled back, and only then is the transaction completely finished when it enters the Aborted state.
- (e) The task of the TP system is to manage the correct and efficient execution of transactions.
-

4

Theory of serialisability

The task of the scheduler within the TP system is to create schedules which allow transactions to be executed concurrently. In this section we have a closer look at these schedules and what it means for them to be correct. This will lead to a discussion of the theory of serialisability as the fundamental theory behind the correct execution of transactions.

4.1 Schedules

We noted that a transaction consists of a number of operations. A **schedule** is a sequence of operations that specifies the chronological order in which operations of concurrent transactions are executed. More specifically, a schedule for a set of transactions must consist of all the operations that make up those transactions, and it must preserve the order in which the operations appear in each individual transaction.

Table 2 shows an example of a schedule for two transactions. The first transaction, T_1 , is the transfer of £100 from account X to account Y that we saw earlier. The other transaction, T_2 , takes 50% off the balance of one account and adds it to another account. Both transactions have the same consistency requirement: the sum of the accounts should remain unchanged.

Table 2 Schedule 1

Step	T_1 transfer £100 from X to Y	T_2 transfer 50% from X to Y
1	$X.read()$	
2	$X = X - 100$	
3	$X.write()$	
4	$Y.read()$	
5	$Y = Y + 100$	
6	$Y.write()$	
7		$X.read()$
8		$value = X * 0.5$
9		$X = X - value$
10		$X.write()$
11		$Y.read()$
12		$Y = Y + value$
13		$Y.write()$

Schedule 1 is a **serial schedule**, because the transactions are executed in strict serial order: first one transaction, then the other transaction. Here the order is first T_1 then T_2 , which we can write as $T_1 < T_2$.

Another serial schedule is shown in schedule 2, this time showing $T_2 < T_1$.

Table 3 Schedule 2

Step	T_1 transfer £100 from X to Y	T_2 transfer 50% from X to Y
1		$X.read()$
2		$value = X * 0.5$
3		$X = X - value$
4		$X.write()$
5		$Y.read()$
6		$Y = Y + value$
7		$Y.write()$
8	$X.read()$	
9	$X = X - 100$	
10	$X.write()$	
11	$Y.read()$	
12	$Y = Y + 100$	
13	$Y.write()$	

We now show schedule 3, which is not serial. In this schedule the operations of the transactions are interleaved.

Table 4 Schedule 3

Step	T_1 transfer £100 from X to Y	T_2 transfer 50% from X to Y
1		$X.read()$
2		$value = X * 0.5$
3		$X = X - value$
4		$X.write()$
5	$X.read()$	
6	$X = X - 100$	
7	$X.write()$	
8		$Y.read()$
9		$Y = Y + value$
10		$Y.write()$
11	$Y.read()$	
12	$Y = Y + 100$	
13	$Y.write()$	

In schedules 1 and 2 we saw serial schedules, and we know that if transactions are executed one after the other then the database will remain in a consistent state.

In schedule 3 there is some form of interleaving of operations, but it is interesting to note that even with this interleaving the database remains consistent. Recall that in the case of a transfer transaction, consistency entails that the sum of both accounts involved in the transfer remains unchanged.

To convince ourselves, we can perform a number of calculations to check that this is true. We assume that account X starts off with a balance of £500, and account Y with a balance of £1000, which gives the following results.

Table 5 Results for schedules 1, 2 and 3

	Balance of account X	Balance of account Y	Sum of balances of accounts X and Y
Initial value	£500	£1000	£1500
Value after schedule 1	£200	£1300	£1500
Value after schedule 2	£150	£1350	£1500
Value after schedule 3	£150	£1350	£1500

Having started with the sum of accounts X and Y of value £1500, we see that all three schedules maintain this. It is important to note that the schedules 1 and 2 do not yield exactly the same results for accounts X and Y , and that is not a problem. As long as the consistency requirement is satisfied, that the total sum of X and Y should remain £1500, the schedules are showing executions that would keep the database consistent.

You should also note that schedule 3 is equivalent to one of the serial schedules, namely schedule 2.

A different schedule showing an interleaved execution of transactions T_1 and T_2 is presented in schedule 4.

Table 6 Schedule 4

Step	T_1 transfer £100 from X to Y	T_2 transfer 50% from X to Y
1		$X.read()$
2		$value = X * 0.5$
3		$X = X - value$
4	$X.read()$	
5	$X = X - 100$	
6	$X.write()$	
7		$X.write()$
8		$Y.read()$
9		$Y = Y + value$
10		$Y.write()$
11	$Y.read()$	
12	$Y = Y + 100$	
13	$Y.write()$	

The schedule presented in schedule 4 does not preserve the requirement that the sum of the accounts involved in a transfer should remain the same. This can be seen in Table 7, where the total sum is now no longer £1500: an amount (£150) has disappeared from the system.

Table 7 Results for schedule 4

	Balance of account <i>X</i>	Balance of account <i>Y</i>	Sum of balances of accounts <i>X</i> and <i>Y</i>
Initial value	£500	£1000	£1500
Value after schedule 4	£250	£1100	£1350

Schedule 4, shown in Table 6, has led to an inconsistent state. The reason for this is that the transaction that had already started allowed its **intermediate state** to be seen by another transaction. That is, transaction T_2 has calculated a new value for account *X* (steps 2 and 3), but has not written it yet (this will happen in step 7). Before it has a chance to write the new value, account *X* is read again but now by a different transaction (in step 4). This goes against the property of isolation that transactions are supposed to exhibit. We return to this issue in a moment.

Schedule 4 also demonstrates what is known as the **lost update problem**. The update made by the write operation of transaction T_1 for account *X* (step 6) is effectively lost, because transaction T_2 overwrites this value with a different one (step 7).

Besides schedules 1 to 4, other interleavings are possible: some of them will leave the database in a consistent state, others will leave it in a inconsistent state. It is not safe to leave the control of the concurrent executions entirely with the operating system, as many inconsistent schedules could then be generated. Instead this task is given to the scheduler within the TP system, which acts as the concurrency control component.

4.2 Serialisability

We noted earlier that one of the desirable properties of a transaction is that it preserves the consistency of the database: before the transaction starts the database is consistent, and after executing the transaction the database is consistent. We note that schedules 1 and 2 both show serial schedules, in which the transactions execute one after the other, in strict serial order. The serial schedules were shown to maintain the consistency of the database.

Schedule 3 was not strictly serial, but we said that it was equivalent to a serial schedule.

This gives the following definition of **serialisability**.

A schedule for a group of concurrent transactions is serialisable if, and only if, it produces the same result as if the transactions had executed in some serial order.

Serialisability is also sometimes referred to as serial equivalence. Note that different serial orders may (or may not) yield different results, but that doesn't matter: what matters is that serial execution will always leave the data consistent.

Note that this is the database definition of serialisability, which is entirely different from the Java `Serializable` objects. (See Unit 7.)

Exercise 2

What is the similarity between the definition of serialisability for transactions, and the definition of thread safety for classes as discussed in *Units 3 and 4*?

Discussion

A class is considered thread-safe if its instances behave under concurrent method calls as if they were called sequentially. This concept is similar to that for the execution of transactions, which should behave as if their execution was serial.

4.3 Conflicting operations

The scheduler, as part of the TP system, has the task of putting together serialisable schedules for the execution of transactions. However, how can we determine whether a schedule generated by the scheduler is of serial equivalence or not?

To begin to answer this question, we note that in the schedules we have shown, the important synchronisation moments are to do with the read and write moments, and that in between these reads and writes, a transaction may be engaged with a number of other steps manipulating the data it holds in main memory. Given that these are held in main memory, and don't involve access to the database, we ignore these steps for our discussion, and concentrate on the read and write operations.

First of all, we note that read and write operations that relate to different data resources do not affect each other. That is, if transaction T_1 reads data object Q , and transaction T_2 writes data object Z , then there is no interference between these operations as they are not at all related. Hence, we concentrate only on operations that involve the same data object.

We present the following definition of so-called **conflicting operations**.

Operations Op_m and Op_n , of transactions T_m and T_n respectively, conflict if, and only if, there exists some data object Q accessed by both Op_m and Op_n and at least one of these operations wrote Q .

Note that T_m and T_n are distinct transactions.

In Table 8 we list the combinations of read and write operations, and indicate whether or not the operations conflict.

Table 8 Conflicting operations

T_m with Op_m	T_n with Op_n	Op_m and Op_n conflict?
$Q.read_m()$	$Q.read_n()$	No
$Q.read_m()$	$Q.write_n()$	Yes
$Q.write_m()$	$Q.read_n()$	Yes
$Q.write_m()$	$Q.write_n()$	Yes

Intuitively, operations conflict if the order in which they are performed affects the result. For operations that do not conflict, it doesn't matter in which order they are performed.

For example, if Q has some defined value, and the schedule shows $Q.read_m()$ (of transaction T_m) followed by $Q.read_n()$ (of transaction T_n), both operations will have read the same value. If the order had been $Q.read_n()$ followed by $Q.read_m()$, then the result would have been exactly the same.

However, if Q has some defined value, and the schedule shows $Q.read_m()$ followed by $Q.write_n()$, then Q may have been given a new value, but transaction T_m has read the old value. Had the order of the two operations been the other way round, $Q.write_n()$ followed by $Q.read_m()$, then transaction T_m would have been working with the new value. So for conflicting pairs of operations, it matters in which order they are performed.

We can use the idea of conflicting operations to determine whether a given schedule shows a serialisable ordering of transactions. This is discussed in the next subsection.

4.4 Testing for serialisability

In order to test whether a given schedule shows a serialisable execution of the transactions, we make use of a graphical notation: **precedence graphs**. Such graphs are a way of representing the order constraints among a collection of transactions. Here we explain how this works.

Suppose we draw a diagram with a node to represent each of several transactions, as shown in Figure 3.

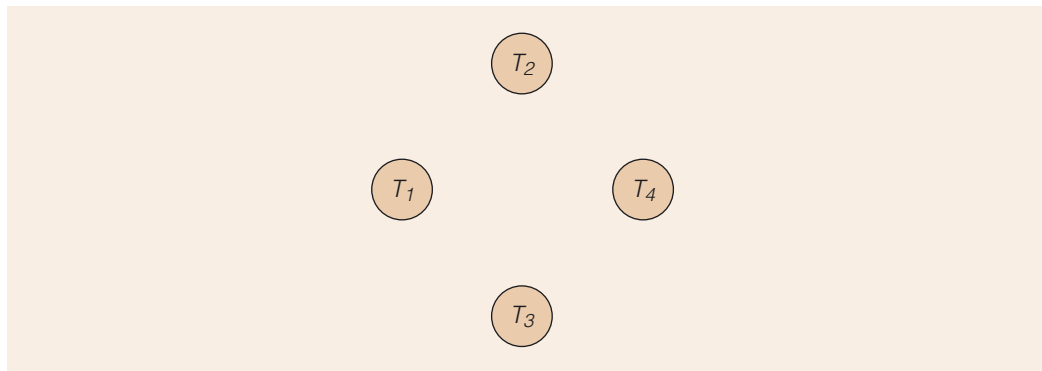


Figure 3 Precedence graph for transactions

Now we examine all the cases where two transactions perform a conflicting pair of operations on a shared data object. If, in the given schedule, transaction T_2 performed its operation before T_3 performed the other operation of the conflicting pair, we draw an arrow (known as an edge) from T_2 to T_3 . This is shown in Figure 4.

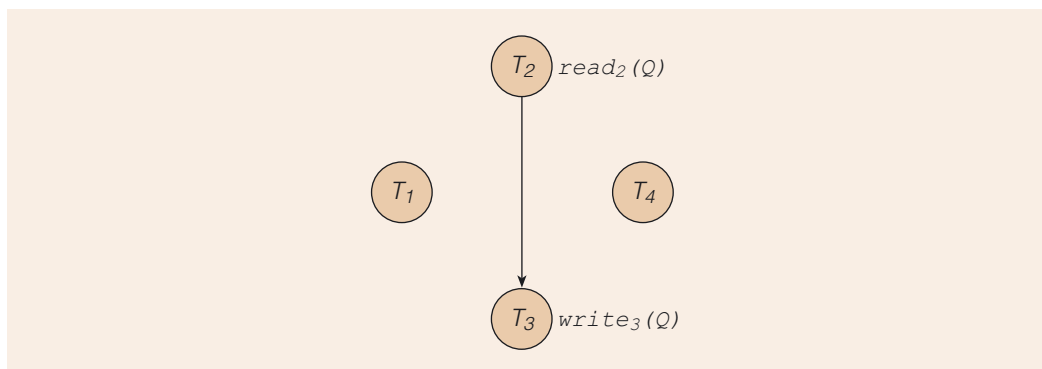
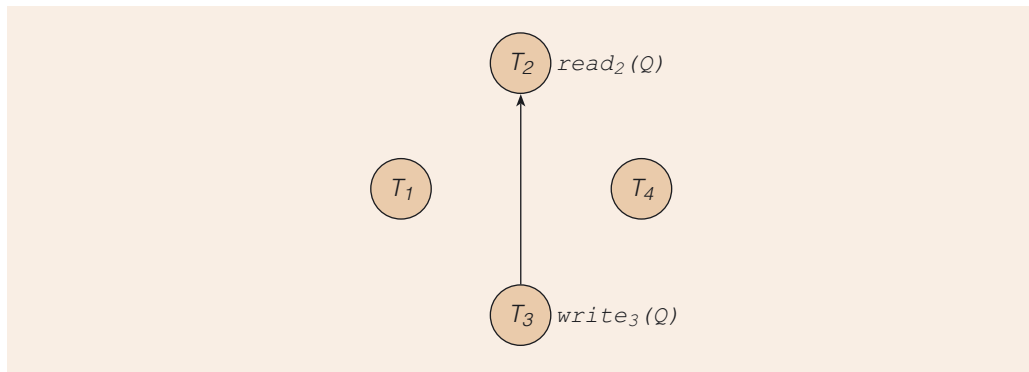


Figure 4 Precedence graph for transactions: $T_2 < T_3$

On the other hand, if T_3 operated on the data object Q before T_2 , we draw an arrow the other way (Figure 5).

Figure 5 Precedence graph for transactions: $T_3 < T_2$

Then a schedule is serialisable if (and only if) when we've drawn all the precedence arrows for all pairs of operations the resulting graph has no cycles – i.e. closed chains of arrows that lead back to their starting point.

A schedule in which one or more pairs of conflicting operations are executed in the order $T_2 < T_3$, while other pair(s) of conflicting operations are executed in the order $T_3 < T_2$, contains a cycle – this is depicted in Figure 6(a). There can also be an indirect cycle, as shown in Figure 6(b), where, for example, pairs of operations execute in the order $T_2 < T_3$, but also some execute in the order $T_3 < T_4$ and $T_4 < T_2$. From the point of view of serialisability no distinction is made between direct or indirect cycles: both are cycles and hence an indication of non-serial equivalence.

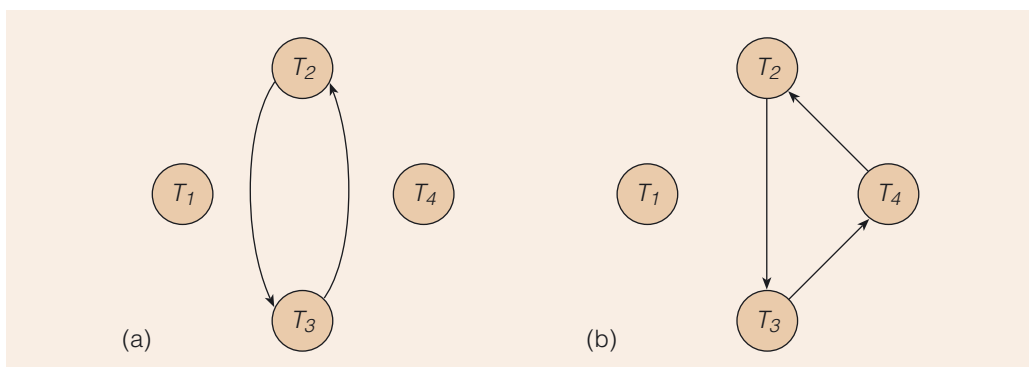


Figure 6 Precedence graph for transactions containing (a) a direct cycle and (b) an indirect cycle

Exercise 3

The following tables show schedules 3 and 4 from those we studied earlier. However, this time we are showing only the read and write operations.

Table 9 Schedule 3 – version 2

Step	T_1 transfer £100 from X to Y	T_2 transfer 50% from X to Y
1		$X.read_2()$
2		$X.write_2()$
3	$X.read_1()$	
4	$X.write_1()$	
5		$Y.read_2()$
6		$Y.write_2()$
7	$Y.read_1()$	
8	$Y.write_1()$	

Table 10 Schedule 4 – version 2

Step	T_1 transfer £100 from X to Y	T_2 transfer 50% from X to Y
1		$X.read_2()$
2	$X.read_1()$	
3	$X.write_1()$	
4		$X.write_2()$
5		$Y.read_2()$
6		$Y.write_2()$
7	$Y.read_1()$	
8	$Y.write_1()$	

For each schedule, list the conflicting operations, and indicate in which order the transactions executed each pair of operations.

Draw the precedence graph for each schedule, and explain whether or not it depicts a serialisable schedule of operations.

Discussion

For schedule 3, the conflicting pairs of operations are as follows.

$X.read_2()$ (step 1) conflicts with $X.write_1()$ (step 4)	$T_2 < T_1$
$X.write_2()$ (step 2) conflicts with $X.read_1()$ (step 3)	$T_2 < T_1$
$X.write_2()$ (step 2) conflicts with $X.write_1()$ (step 4)	$T_2 < T_1$
$Y.read_2()$ (step 5) conflicts with $Y.write_1()$ (step 8)	$T_2 < T_1$
$Y.write_2()$ (step 6) conflicts with $Y.read_1()$ (step 7)	$T_2 < T_1$
$Y.write_2()$ (step 6) conflicts with $Y.write_1()$ (step 8)	$T_2 < T_1$

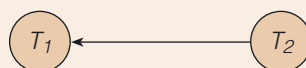


Figure 7 A precedence graph for schedule 3

For schedule 4, the conflicting pairs of operations are as follows.

$X.read_2()$ (step 1) conflicts with $X.write_1()$ (step 3)	$T_2 < T_1$
$X.read_1()$ (step 2) conflicts with $X.write_2()$ (step 4)	$T_1 < T_2$
$X.write_1()$ (step 3) conflicts with $X.write_2()$ (step 4)	$T_1 < T_2$
$Y.read_2()$ (step 5) conflicts with $Y.write_1()$ (step 8)	$T_2 < T_1$
$Y.write_2()$ (step 6) conflicts with $Y.read_1()$ (step 7)	$T_2 < T_1$
$Y.write_2()$ (step 6) conflicts with $Y.write_1()$ (step 8)	$T_2 < T_1$



Figure 8 A precedence graph for schedule 4

For schedule 3, all the pairs of conflicting operations have an ordering of $T_2 < T_1$, hence the precedence graph is shown with an edge going from T_2 to T_1 . It is serialisable.

However, schedule 4 has four pairs of conflicting operations in the order $T_2 < T_1$ and two pairs in the order $T_1 < T_2$. Therefore the precedence graph shows both directions, and contains a cycle. This means that the schedule is not serialisable.

A more complex schedule

The above example was relatively simple. Here we show an example of a schedule that contains a few more transactions and their objects, and we will ask you to draw its associated precedence graph.

Table 11 Schedule for transactions T_1 to T_5 , using data objects U, V, W, X, Y and Z

Time step	T_1	T_2	T_3	T_4	T_5
1			$X.read_3()$		
2	$Y.read_1()$				
3	$U.read_1()$				
4	$Z.read_1()$				
5		$V.read_2()$			
6		$W.read_2()$			
7		$V.write_2()$			
8			$Y.read_3()$		
9			$Y.write_3()$		
10					$Z.write_5()$
11	$U.read_1()$				
12				$Y.read_4()$	
13				$Y.write_4()$	
14				$Z.read_4()$	
15				$Z.write_4()$	
16	$U.read_1()$				
17	$U.write_1()$				

Exercise 4

- (a) Find the pairs of conflicting operations in Table 11. Record each pair in a table, making a note of the time step at which each operation occurs, and noting the order of the transactions. We show the first pair of operations to get you started.

Table 12 Template table for conflicting operations for the schedule for transactions T_1 to T_5 , using data objects U, V, W, X, Y and Z

Operation	At time step	Conflicts with operation	At time step	Order of transactions
$Y.read_1()$	2	$Y.write_3()$	9	$T_1 < T_3$
....				

- (b) Using the information from part (a) about the order of the transactions, draw the precedence graph for the schedule of operations.

Discussion

- (a) The pairs of conflicting operations for the schedule in Table 11 are as follows.

Table 13 Conflicting operations for the schedule for transactions T_1 to T_5 , using data objects U, V, W, X, Y and Z

Operation	At time step	Conflicts with operation	At time step	Order of transactions
$Y.read_1()$	2	$Y.write_3()$	9	$T_1 < T_3$
$Y.read_1()$	2	$Y.write_4()$	13	$T_1 < T_4$
$Z.read_1()$	4	$Z.write_5()$	10	$T_1 < T_5$
$Z.read_1()$	4	$Z.write_4()$	15	$T_1 < T_4$
$Y.read_3()$	8	$Y.write_4()$	13	$T_3 < T_4$
$Y.write_3()$	9	$Y.read_4()$	12	$T_3 < T_4$
$Y.write_3()$	9	$Y.write_4()$	13	$T_3 < T_4$
$Z.write_5()$	10	$Z.read_4()$	14	$T_5 < T_4$
$Z.write_5()$	10	$Z.write_4()$	15	$T_5 < T_4$

- (b) The precedence graph for the schedule in Table 11, which involves five transactions and six different objects, is shown below.

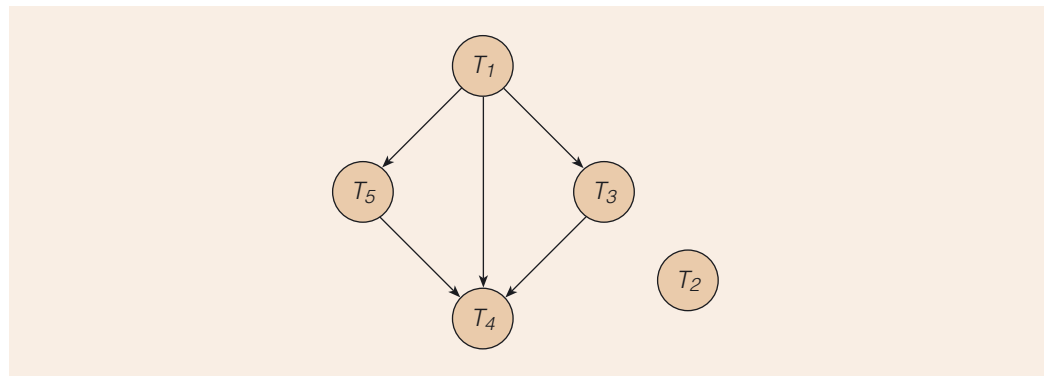


Figure 9 Precedence graph for schedule shown in Table 11

Transaction T_2 does not have any operations that conflict with other transactions, so there are no edges drawn from the other transactions to T_2 .

Precedence graphs and topological sorting

For the precedence graph in Figure 9 it is not entirely straightforward to work out whether or not it contains a cycle, and we need a method to convince ourselves that the precedence graph in Figure 9 is acyclic. This can be done by checking whether a **topological sorting** of the precedence graph gives a linear path through the graph.

Informally, a topological sort is a list of the nodes of a **directed acyclic graph (DAG)** in which each node comes before all nodes to which it has edges. Every DAG has at least one topological sort, and may have many. From this it follows that if we are able to create a topological sort for a directed graph, the graph is shown to be acyclic.

The graph involving just the four transactions T_1 , T_3 , T_4 and T_5 can be sorted in the following two ways.

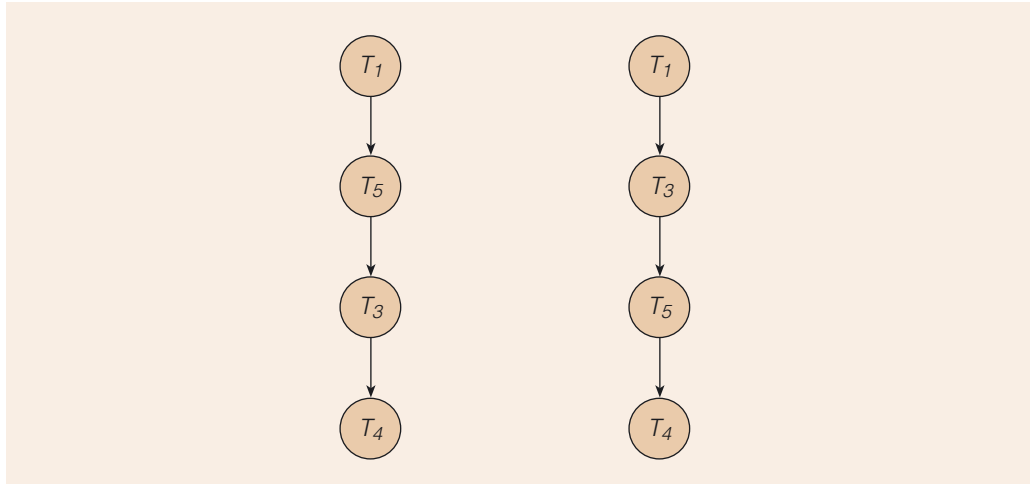


Figure 10 Topological sort for the precedence graph of Figure 9

The edges between the nodes reflect the ordering of the transactions. Note that the existence of just one topological sort would have been sufficient to indicate that we are dealing with a DAG – which in turn indicates that the given schedule the DAG represents, is serialisable.

There are a number of these cycle-detection algorithms, such as depth-first search, which are described in books for algorithms or graph theory. In this course we will not further concern ourselves with the details of such algorithms, except to say that they exist, and that they are not very efficient because they require in the order of n^2 operations, where n is the number of transactions.

Back to serialisability

Our original question concerned how we can determine whether a schedule generated by the scheduler is serialisable or not. We can summarise the above points by saying that in order to establish this, a precedence graph must be created of the schedule, based on the conflicting operations between the transactions in the schedule. Then a cycle-detection algorithm should be applied, such as we showed for topological sorting, in order to detect whether the given precedence graph is serialisable.

Obviously the scheduler would not use such a brute force algorithm to generate the schedules in the first place, but this approach serves as a means for us to understand which schedules are serialisable and which are not. In the next section we will look at some of the algorithms used by the scheduler to generate such schedules.

This brings to a conclusion our discussion of the theory of serialisability which is fundamental to transaction processing. Serialisability is the most widely accepted correctness criterion for concurrency control algorithms.

Note that ‘directed’ in DAG refers to the fact that there is a definite direction from one node to the next, and we cannot traverse the edge in the opposite direction. Hence the figure is drawn with arrows, rather than lines.

Exercise 5

A class `BankAccount` (as discussed in Subsection 3.3) has a number of operations as described previously (repeated in Table 14).

Table 14 Operations for class `BankAccount`

Operation	Description
<code>credit(amount)</code>	Adds the amount to the balance stored by the object.
<code>debit(amount)</code>	Takes away the amount from the balance.
<code>getBalance()</code>	Reads the current balance and returns its value.

It is possible to work out whether or not these operations conflict with each other, using a similar approach to the one used for the operations *read* and *write* in the previous tables. For this exercise, we assume that `debit` and `credit` do not conflict with each other, since the order in which they are carried out (i.e. a `credit` before a `debit`, or a `debit` before a `credit`) does not matter.

- Using the descriptions of the methods, indicate which `BankAccount` operations conflict with each other.
- Assume that we have two transactions T_1 and T_2 , and three `BankAccount` objects `customerAcc`, `shopAcc` and `distributorAcc`. The transactions are specified as follows:
 - ▶ T_1 debits £250 from the customer's account and credits it to the shop's account;
 - ▶ T_2 adds interest to the three accounts in the order: customer account, shop account and distributor account.

Below we show a schedule with a possible interleaving of the operations.

Table 15 Schedule with interleaved `BankAccount` operations

Time step	Transaction T_1	Transaction T_2
1		<code>customerAcc.addInterest()</code>
2	<code>customerAcc.debit(250)</code>	
3		<code>shopAcc.addInterest()</code>
4		<code>distributorAcc.addInterest()</code>
5	<code>shopAcc.credit(250)</code>	

Using your answer from part (a), give the conflicting pairs of operations for this schedule, and indicate for each pair the order in which the transactions act.

- Show the precedence graph for the schedule in part (b) and explain whether this is a serialisable schedule.

Discussion

(a) The conflicting operations are as follows.

Table 16 Conflicts between BankAccount operations

Name of operation	Conflicts with
credit	getBalance, addInterest
debit	getBalance, addInterest
addInterest	debit, credit, getBalance

For example, `credit` conflicts with `getBalance`, as `getBalance` would obtain a different reading of the balance depending on whether the `credit` had already taken place. Similarly, `addInterest` would yield a different result depending on whether it came before or after a `credit` operation.

(b) There are two pairs of conflicting operations:

T_1 :debit and T_2 :addInterest operating on customerAcc object ($T_2 < T_1$)

T_1 :credit and T_2 :addInterest operating on shopAcc object ($T_2 < T_1$)

(c) The precedence graph for the schedule is shown.



Figure 11 Precedence graph for the schedule

The graph does not have a cycle, and we can conclude that the schedule is serialisable.

SAQ 2

- What is a serial schedule?
- Give the definition of serialisability.
- When executing a number of transactions, several serial schedules may be possible. These serial schedules do not necessarily all give the same final results for the data objects involved. Explain whether or not you think this is problematic.
- What are conflicting operations?
- How is the notion of conflicting operations used to determine whether a schedule is serialisable?
- Precedence graphs can be used as a notation for transactions and the conflicting operations of those transactions. Explain under which circumstances such graphs could contain a cycle.

ANSWER.....

- A serial schedule is one in which transactions execute strictly one after the other, without any interleaving of operations.
- A schedule for a group of transactions is serialisable if, and only if, it produces the same results as if the transactions had executed in some serial order.
- This is not problematic, as the main concern is whether the data objects are in a consistent state.
- Operations conflict if the order in which they are performed affects the result.
- The notion of conflicting operation allows us to determine in what order transactions execute a pair of conflicting operations. Then, if for all the pairs of conflicting operations in a schedule the order of execution by transactions is the same, we know that the schedule is equivalent to a serial schedule.
- A precedence graph contains a cycle if the schedule it represents is not serialisable.

5

Concurrency control for transactions

In Section 3 we stated that isolation is a fundamental property of transactions. In Section 4 we discussed that if several transactions are executed concurrently, the isolation property may be in danger of being compromised, and that it is important to make sure that the schedules of operations of multiple transactions are serialisable.

In this section you will examine algorithms for transaction processing that ensure that the schedules are serialisable. The problem of failure is ignored in this section, and will be discussed in Section 6.

We will discuss two classes of concurrency control mechanism: **pessimistic algorithms** and **optimistic algorithms**. Pessimistic algorithms assume that there are likely to be conflicts and they take measures to avoid them, whereas optimistic algorithms assume that conflicts are likely to be rare, and can therefore be dealt with after the event.

Depending on the nature of the application we are writing, one algorithm may be more suitable than another. If we expect low contention for the resources being accessed, then it's cheapest to employ an optimistic strategy and sort things out retrospectively in the few cases where there is conflict.

The algorithms employ different strategies for detecting and handling conflicts between transactions. We will look at three methods: two-phase locking, a pessimistic algorithm based on locking data objects; time-stamp ordering, another pessimistic algorithm, but based on ordering the transactions; and optimistic concurrency control, an optimistic algorithm based on locking. Note that there are also other methods that we will not discuss here as these are not commonly used and are not available in commercial applications.

5.1 Two-phase locking

The basic idea behind locking is that a lock gets set on an object before it is accessed by a transaction. Only the holder of the lock, i.e. the transaction, can invoke an operation on the object. Transactions typically deal with several objects; hence several objects may need to be locked. One way of organising this is to require that a transaction gets hold of all the objects at the start of the transaction, locks them, then releases them on commit or abort. However, this may not achieve the best level of concurrency, since the transaction may be able to make progress on some objects that are not under contention from other transactions. In this section we discuss a method called two-phase locking which controls access to shared objects while achieving a better degree of concurrency.

The overall aim of the **two-phase locking (2PL)** algorithm is to ensure that the schedules for transactions are such that the effects of the transactions on the shared data are serially equivalent. By serialising the *access to the objects*, we can achieve serial equivalence of the transactions.

We show this with an example involving three transactions T , U and V , and four data objects A , B , C and D :

Transaction T : *start*, $A.read()$, $B.write()$, $D.write()$, *commit*;

Transaction U : *start*, $B.write()$, $C.write()$, *commit*;

Transaction V : *start*, $C.write()$, $D.write()$, *commit*;

for which the following interleaved schedule would be possible if 2PL were to be used.

Table 17 Two-phase locking for transactions T , U and V

Step	T	U	V
1	<i>start</i>		
2		<i>start</i>	
3			<i>start</i>
4	$A.lock()$		
5	$A.read()$		
6			$C.lock()$
7		$B.lock()$	
8			$C.write()$
9		$B.write()$	
10			$D.lock()$
11			$D.write()$
12			$C.unlock()$
13		$C.lock()$	
14		$C.write()$	
15			$D.unlock()$
16		$B.unlock()$	
17	$B.lock()$		
18	$B.write()$		
19	$D.lock()$		
20	$D.write()$		
21	$A.unlock()$		
22		$C.unlock()$	
23		<i>commit</i>	
24	$B.unlock()$		
25			<i>commit</i>
26	$D.unlock()$		
27	<i>commit</i>		

In Table 17 we see that before an object is first used by a transaction, it is locked (e.g. steps 4 and 5). Furthermore, we note that a transaction cannot make use of an

object which is locked by another transaction, and must wait until the other transaction has released the lock for the object (e.g. transactions *T* and *U* with object *B*, in steps 16 and 17).

Two-phase locking derives its name from the fact that there are two phases: the **acquire phase** and the **release phase**.

- ▶ In the acquire phase, the transaction acquires objects, *but does not release any*.
- ▶ In the release phase, the transaction unlocks objects, *but does not acquire any*.

In the acquire phase, a transaction is gradually acquiring the locks it requires, as and when it needs them, until it has the total requirement for all its locks. In the release phase, it unlocks the locks. There are two strategies for releasing them.

- ▶ The **strict 2PL** form releases all locks on commit. That way there is no doubt that the transaction was successful. One could think of strict 2PL in terms of a transaction collecting all the locks it needs in a shopping basket, and releasing them all in one go at the till (at which point the transaction is not allowed to add any more items to the basket).
- ▶ In the non-strict form, known simply as 2PL, a transaction can let go of the locks gradually, after it has finished using them. However, it is still the case that it is not allowed any new locks after it has started releasing locks, otherwise there would be no guarantee that all conflicting pairs of operations had been executed in the same order.

Exercise 6

Table 17 shows a possible interleaved execution of two transactions using two-phase locking. For each transaction, identify the two phases in terms of the steps listed.

Discussion

For transaction *T*:

- ▶ the acquire phase is from step 1 to step 20;
- ▶ the release phase is from step 21 to step 27.

For transaction *U*:

- ▶ the acquire phase is from step 2 to step 15;
- ▶ the release phase is from step 16 to step 23.

For transaction *V*:

- ▶ the acquire phase is from step 3 to step 11;
- ▶ the release phase is from step 12 to step 25.

Note that the release phase starts as soon as the first lock is released – up until that point we are dealing with the acquire phase.

Granularity of locks

An important design issue for locking systems is the **granularity of the locks**. In the banking example we have assumed that locks are allocated for each `BankAccount` object, which seems about right. Another option might have been to lock only part of each `BankAccount` object, say the part that refers to the account's balance, and not to the fields that refer to the customer's personal details.

Different types of lock: read and write

Another issue that affects the concurrency is the precise nature of the locks. Operations differ in whether they are in essence reading or writing operations, and if a number of transactions require access to a shared object purely for reading purposes

(and no transactions are going to write to the object) we should consider allowing these transactions to *share the lock*.

For example, in the context of `BankAccount` objects, there would be no problem if several transactions attempted to use `getBalance` at the same time because it is purely a read operation.

We could thus distinguish between **write locks** and **read locks**, also referred to as **exclusive locks** and **shareable locks**. If these two types of lock are used we require additional rules regarding the setting of locks.

- ▶ If a transaction wishes to acquire a read lock for an object it can do so only if no other transaction currently holds a write lock for that object.
- ▶ If a transaction wishes to acquire a write lock for an object it can do so only if no other transaction currently holds a read lock or a write lock for that object.

The releasing of locks remains the same: a transaction either releases all its locks upon commit, or it could release them piecemeal when it has finished with them.

There is an interesting issue to do with the case where a transaction has requested a *read* lock and obtained it, but then later on needs to *write* to that object. In that case it must seek to acquire the write lock like any would-be writer.

Deadlock in two-phase locking

Using locks has the disadvantage that the system is likely to run into problems of deadlock. A transaction may have acquired one lock and be waiting to obtain the next lock. However, the next lock may be held by another transaction which is waiting to obtain the lock held by the first transaction.

We recall from *Unit 3* that four conditions must hold in order for deadlock to occur. The first three are policies.

- 1 Exclusive access – shared objects can be accessed by only one process at a time.
- 2 Hold while waiting – a process can retain a lock on objects while waiting to acquire other objects.
- 3 No pre-emption – a process cannot be forced to give up objects against its will.

If these all hold, then it is possible for a circular wait to develop, giving the fourth condition.

- 4 Circular wait – a cycle of two or more processes exists in which each is waiting for a resource that has been locked by the next process in the chain.

If, and only if, all these conditions apply there will be deadlock. In order to break a deadlock, or prevent it from occurring in the first place, we must arrange things so that at least one of the four conditions doesn't apply. Some of the common measures are as follows.

- 1 Exclusive access can be relaxed in some cases by using shareable locks, as we have already discussed. Shared locks will reduce the likelihood of deadlock.
- 2 Hold while waiting may be replaced by a policy in which a transaction must acquire all the locks it will need in a single atomic action right at the start. However, this may restrict concurrency unnecessarily; moreover it may not be possible for the transaction to specify at the start exactly which objects are going to be needed.
- 3 No pre-emption may be relaxed by allowing the system to detect deadlocks and resolve them by forcing transactions to relinquish resources. Transactions may also be made to give up locks by the use of timeouts. When a transaction takes out a lock it has a period of time during which no other transaction can gain the lock, but once

that time has expired it must surrender the lock if another transaction wants to acquire it.

- 4 Circular wait can be prevented from occurring in various ways, although we do not discuss them here.

When a timeout occurs, a transaction can attempt to regain the lock, or, if this is not possible, it may have to abort. Although timeouts are the most common way of recovering from deadlock, they have their own problems. To start with, it is very difficult to decide what would be an appropriate length of time to set – and this may differ from one system or application to the next. Furthermore, timeouts can lead to the interruption of transactions that were not actually deadlocked. If the system is handling a lot of transactions, and the system is overloaded, it may simply take longer for a transaction to complete. It may also be the case that transactions that take longer to complete due to their nature, are penalised unnecessarily.

5.2 Time-stamp ordering

The use of time-stamps is one example of the importance of time for large systems. In *Unit 9* we will discuss in more detail the role of time in distributed systems.

Time-stamp ordering (TSO) is another pessimistic algorithm to manage concurrency control. It works by assigning each transaction a unique time-stamp, usually when the transaction starts. The system enforces that pairs of conflicting operations can only be executed in the order of the time-stamps. This prevents there being any cycles, thus ensuring serialisability, as we will explain shortly.

There are two ways to implement the time-stamp scheme:

- 1 base the time-stamp on the value of the system clock at the time the transaction enters the system;
- 2 base the time-stamp on a logical counter, which is incremented after a new time-stamp is assigned to a transaction.

In simplified form, the time-stamp ordering is enforced as follows. Each object keeps a time-stamp, which is the one associated with the transaction that accessed it most recently. If another transaction now requests an operation that conflicts with one that has been carried out previously on this object, the time-stamp of the transaction is compared with the time-stamp recorded by the object. There are two alternatives.

- ▶ If the time-stamp of the transaction is *not* earlier than the time-stamp recorded by the object, so that the order of the time-stamps is being followed, all is well. The operation will be successful and the object will update its time-stamp to that of the more recent transaction to access the object.
- ▶ If the transaction's time-stamp *is* earlier than the one recorded by the object, the operation is not allowed to proceed. The transaction must abort and restart. In due course it can re-attempt the operation, but now with the later time-stamp it will have been given when it restarted.

Example 1

To illustrate time-stamp ordering, we first consider a very simple scenario. A transaction *T* has time-stamp 29, a transaction *U* has time-stamp 30, and the operations listed in Table 18 are presented to the scheduler.

Table 18 Schedule using TSO – Schedule I

Step	<i>T</i> (time-stamp 29)	<i>U</i> (time-stamp 30)
1	<i>A.read()</i>	
2	<i>B.read()</i>	
3		<i>B.write()</i>
4		<i>A.write()</i>

This schedule is possible under time-stamp ordering. In step 2 object *B* acquires time-stamp 29, so when the conflicting *write* operation in step 3 is presented, the *write* operation is accepted because it has a time-stamp (30) that is higher than object *B* currently has. The same applies for the conflicting operations on object *A*. However, if the operations were presented in the order shown in Table 19, the situation would be different.

Table 19 Schedule using TSO – Schedule II

Step	<i>T</i> (time-stamp 29)	<i>U</i> (time-stamp 30)
1		<i>B.write()</i>
2		<i>A.write()</i>
3	<i>A.read()</i>	
4	<i>B.read()</i>	

This schedule would not be possible under TSO because object *A* has a higher time-stamp (30) than the *read* operation (time-stamp 29) at the time it is presented in step 3. Hence the above schedule would not be allowed under TSO: transaction *T* would abort, and only transaction *U* is able to complete.

Example 2

We will now examine in detail a more interesting example involving three transactions *T*, *U* and *V*, and three data objects *A*, *B* and *C*:

Transaction *T*: *start*, *A.write()*, *B.read()*, *commit*;

Transaction *U*: *start*, *B.write()*, *C.write()*, *commit*;

Transaction *V*: *start*, *A.read()*, *B.read()*, *B.write()*, *C.write()*, *commit*.

Suppose the transactions are started in the order we presented them: *T*, then *U* and then *V*. Hence transaction *T* is given time-stamp 1, *U* is given time-stamp 2, and *V* is given time-stamp 3.

Table 20 presents an imaginary transaction processing log, for one possible interleaved schedule of the transactions. For the sake of this example the data objects start with no time-stamp.

Table 20 TSO log

Entry	Action	Result
1	<i>T: start</i>	Transaction <i>T</i> starting with time-stamp 1.
2	<i>T: A.write()</i>	Transaction <i>T</i> time-stamp 1 invoking <i>write()</i> on data object <i>A</i> : data object <i>A</i> acquiring time-stamp 1.
3	<i>U: start</i>	Transaction <i>U</i> starting with time-stamp 2.
4	<i>V: start</i>	Transaction <i>V</i> starting with time-stamp 3.
5	<i>U: B.write()</i>	Transaction <i>U</i> time-stamp 2 invoking <i>write()</i> on data object <i>B</i> : data object <i>B</i> acquiring time-stamp 2.
6	<i>V: A.read()</i>	Transaction <i>V</i> time-stamp 3 invoking <i>read()</i> on data object <i>A</i> with time-stamp 1. Time-stamp of <i>V</i> is <i>not earlier</i> than time-stamp of <i>A</i> : operation successful. Data object <i>A</i> acquiring time-stamp 3.
7	<i>T: B.read()</i>	Transaction <i>T</i> time-stamp 1 invoking <i>read()</i> on data object <i>B</i> with time-stamp 2. Time-stamp of <i>T</i> <i>earlier</i> than time-stamp of <i>B</i> : operation failed. ***Transaction <i>T</i> aborting*** Time-stamp of data object <i>B</i> unchanged, remains 2. Transaction <i>V</i> which had read object <i>A</i> (log entry 6) needs to be aborted because it read a value that was written by the now aborted transaction <i>T</i> (log entry 2). ***Transaction <i>V</i> aborting***
8	<i>U: C.write()</i>	Transaction <i>U</i> time-stamp 2 invoking <i>write()</i> on data object <i>C</i> : data object <i>C</i> acquiring time-stamp 2.
9	<i>U: commit</i>	Transaction <i>U</i> committed.

The transaction processing log shows that transaction *T* with time-stamp 1 had to be aborted when it came to the operation *B.read()*. This is because object *B* had already been accessed, by a transaction with a higher time-stamp. Furthermore, as a result of transaction *T* aborting, transaction *V* now also has to abort because it had used object *A*, which had been written to by transaction *T*. Transaction *U* is the only transaction able to commit. The other two transactions, *T* and *V*, would be restarted with higher time-stamps.

The algorithm, as we have explained it, is simplified and does not fully explore exactly what data is recorded and hence how it is known that transaction *V* has to be aborted.

Some advantages of TSO:

- It is highly efficient and has fewer deadlock problems than locking.
- It is a simple mechanism and is good for overall system performance. There are few overheads, as the information can be stored locally with each object, and there is no need for any central managing process to look after it.

Some disadvantages of TSO:

- ▶ It can suffer from high levels of aborts. The reason is that, since isolation is not enforced, other transactions may have read a transaction's intermediate state and based their calculations on it. If the transaction now aborts for some reason, those other transactions are invalidated, and a **cascade of aborts** may result.
- ▶ It may be seen as more restrictive than 2PL. There are scenarios where the strict application of the rule means that sometimes transactions are aborted that were not actually in danger of causing problems. (See Exercise 7 parts (a), (b) and (c) for a discussion of such a scenario.)

Exercise 7

- (a) Table 17 (in Subsection 5.1) shows a possible interleaved execution of three transactions using two-phase locking. Convert the schedule in Table 17 so that TSO is used instead of 2PL. Give your answer in the form of a TSO log, similar to that in Table 20.
- (b) Give the precedence graph for the schedule in Table 17 where 2PL was used, and the precedence graph for the schedule in the log you have given as the answer to part (a) of this exercise. If an operation was attempted but then failed, leading to aborting the transaction, you may use dashed lines in the graph to show the precedence between two transactions.
- (c) Using your answers to parts (a) and (b), explain why TSO is said to be stricter than 2PL.
- (d) What is isolation?
- (e) Why do we say that TSO does not enforce the property of isolation? Illustrate your answer with an example scenario involving transactions *T*, *U* and *V*.

Discussion

- (a) The log for the schedule shown in Table 17 would be as follows.

Table 21 TSO log for converted schedule

Action	Result
<i>T</i> : start	Transaction <i>T</i> starting with time-stamp 1.
<i>U</i> : start	Transaction <i>U</i> starting with time-stamp 2.
<i>V</i> : start	Transaction <i>V</i> starting with time-stamp 3.
<i>T</i> : <i>A.read()</i>	Transaction <i>T</i> time-stamp 1 invoking <i>read()</i> on data object <i>A</i> : data object <i>A</i> acquiring time-stamp 1.
<i>V</i> : <i>C.write()</i>	Transaction <i>V</i> time-stamp 3 invoking <i>write()</i> on data object <i>C</i> : data object <i>C</i> acquiring time-stamp 3.
<i>U</i> : <i>B.write()</i>	Transaction <i>U</i> time-stamp 2 invoking <i>write()</i> on data object <i>B</i> : data object <i>B</i> acquiring time-stamp 2.
<i>V</i> : <i>D.write()</i>	Transaction <i>V</i> time-stamp 3 invoking <i>write()</i> on data object <i>D</i> : data object <i>D</i> acquiring time-stamp 3.
	(table continued overleaf)

Action	Result
$U: C.write()$	Transaction U time-stamp 2 invoking $write()$ on data object C with time-stamp 3. Time-stamp of U is <i>earlier</i> than time-stamp of C : operation failed. ***Transaction U aborting*** Time-stamp of data object C unchanged, remains 3.
$T: B.write()$	Transaction T time-stamp 1 invoking $write()$ on data object B with time-stamp 2. Time-stamp of T is <i>earlier</i> than time-stamp of B : operation failed. ***Transaction T aborting*** Time-stamp of data object B unchanged, remains 2.
$V: commit$	Transaction V committed.

(b) The precedence graphs are as follows.

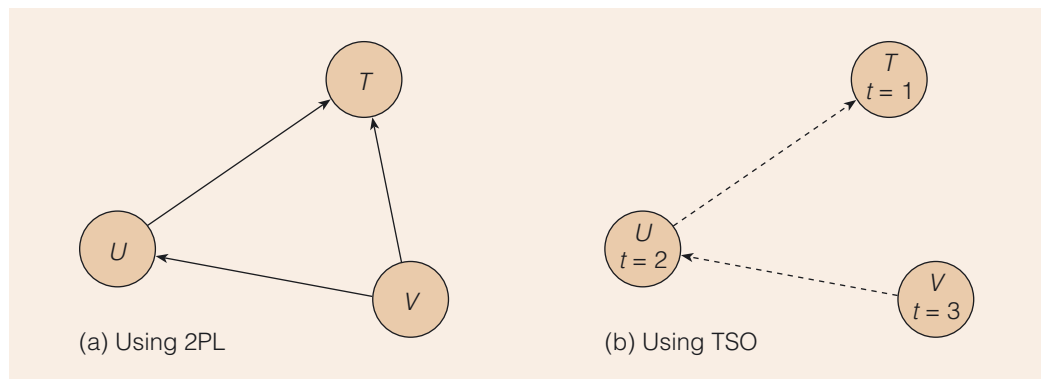


Figure 12 Precedence graphs for transactions T , U and V using (a) 2PL and (b) TSO

(c) The log in part (a) showed that a schedule that was correct for 2PL leads to an aborting of two transactions when TSO is used. This is to do with the way the transactions acquire their time-stamp in TSO, which predefines the ordering of transactions statically at the beginning of processing.

The same is shown in the precedence graphs, where in graph (a) we see that the schedule was serialisable, as there are no cycles in the graph. In graph (b) we note that operations were rejected (leading to aborting transactions) even though there are no cycles in the graph.

Both the log and the comparison of the two precedence graphs indicate that TSO is stricter than 2PL and can disallow a schedule that is serialisable.

(d) Isolation means that the intermediate results of parts of a transaction should not be revealed until the transaction completes.

(e) With TSO, once an operation has been carried out, its results are visible (i.e. the objects involved are locked only for the duration of the operation). In Table 20, we see that in log entry 6 transaction V reads a value that was written by a transaction (T) that has not yet committed. This is an example of isolation not being strictly enforced.

We also note that in Table 20, log entry 7, transaction V has to abort as a result of transaction T having had to abort. This is known as the problem of cascading aborts.

5.3 Optimistic concurrency control

The main goal of **optimistic concurrency control** is to avoid the overheads and problems associated with locking mechanisms, such as lock management, deadlock detection or prevention, and the potential for reduced concurrency resulting from measurements to avoid cascading aborts (not discussed). The optimistic approach works better for applications where the likelihood of clients accessing the same data is very low, and where fast response is important, such as in interactive applications.

The optimistic approach is based on the observation that in most applications the likelihood that two clients want to access the same data is minimal. Transactions are allowed to proceed as if there is no possibility of conflict, and only when a transaction requests a `closeTransaction` operation is a check performed. If it turns out that there is a conflict, one or more transactions are aborted.

Each transaction has the following phases.

- ▶ **Working phase** – the transaction works with ‘tentative’ version of objects, where ‘tentative’ means a copy of the most recently committed version of the object.
- ▶ **Validation phase** – when `closeTransaction` is issued, the check for conflicts is carried out. If validation succeeds, the transaction is allowed to commit. If validation fails, either this transaction or some other transaction will need to be aborted.
- ▶ **Update phase** – if the transaction is validated, the tentative objects that the transaction has worked on will be made permanent.

There are a number of different ways that validation in the validation phase can be carried out. These are based on associating time-stamps with the transactions at the moment they enter the validation phase, and using read–write conflict rules to ensure that the scheduling of a particular transaction is serially equivalent with all other overlapping transactions, i.e. the transactions that had not yet committed at the time this transaction started. However, we will not study those further.

5.4 Comparison of methods for concurrency control

We have discussed three main methods for concurrency control. TSO and 2PL are both pessimistic in their approach, and aim to detect the conflicts between transactions as early as possible. However, there are also some differences.

- ▶ Type of conflict: in 2PL the conflict is about the contention for locks, which, once acquired, will avoid any possibility of conflicting operations occurring; whereas in TSO it is about the detection of potentially conflicting operations.
- ▶ Ordering of the transactions: TSO is static and 2PL is dynamic. That is, TSO decides on an order at the start of a transaction, but 2PL decides this dynamically, according to the order in which the locks are acquired.
- ▶ Handling of conflicts between transactions: TSO aborts the transaction that is too late (and possibly others) *immediately*, whereas 2PL avoids the possibility of conflicts by using locks. On the other hand, 2PL may give rise to deadlock due to contention for locks, and if a timeout is used to avert the deadlock problems, a transaction that cannot acquire a lock may end up being aborted *eventually*.

When optimistic concurrency control is used, the algorithm is particularly efficient if there are few conflicts. However, a substantial amount of work has to be repeated when a transaction aborts, so this approach is useful for only certain applications.

SAQ 3

- (a) What are the two phases of two-phase locking?
- (b) What is a shared lock and why might we want to use shared locks?
- (c) What is a simple alternative to deadlock detection?
- (d) What are cascading aborts and what causes them?
- (e) What are the advantages and disadvantages of TSO?
- (f) Why is the optimistic concurrency control approach so named?

ANSWER.....

- (a) The two phases are acquire and release. In the acquire phase, the transaction gradually gets hold of all the locks it needs, as and when it needs them, but does not release any locks. In release, the transaction acquires no more locks but lets go of the ones it holds when it has completed work on the locked objects.
 - (b) A shared lock is useful when non-conflicting operations are using the same object, and it is therefore safe for both to use the object. If a shared lock can be used, it increases concurrency.
 - (c) Timeouts.
 - (d) Cascading aborts occur when one transaction has to abort and this leads to other transactions having to abort. This is bad for the performance of a system, as much work may have to be repeated.
 - (e) The advantages of TSO are:
 - ▶ simple and efficient implementation;
 - ▶ concurrency control data is held with individual objects and not centrally;
 - ▶ objects are not locked for longer than the duration of an operation and so circular waits and hence deadlocks cannot occur.The main disadvantages of TSO are:
 - ▶ the loss of flexibility, in that it imposes one serial schedule and excludes other possibilities;
 - ▶ it is more susceptible to transaction aborts than other approaches, due to lack of isolation, and may possibly lead to cascading aborts.
 - (f) This approach is called optimistic because it expects success, rather than failure. That is, it works away, confident that things will probably turn out for the best, and only performs a final check at the very end.
-

6

Crash resilience and data recovery

One of the major realities of distributed systems is that various parts of the system can fail at any point. A server machine can go down just as a client is communicating with the server, or the client itself can fail; the communications may fail or the communication lines can be overloaded to such an extent that the message cannot be delivered. For all these reasons, distributed systems are designed with the possibility of failure in mind.

It is also possible that there is some form of media failure, by which we mean that the storage device to store the database fails in some way, perhaps due to hardware faults or because of problems with the operating system. This could affect a stand-alone machine, of course, and even there we would aim for crash recovery where possible.

An important aspect of system design is the extent to which applications are offered support in order to recover from a system crash. With **crash resilience** we mean that the system is able to continue processing by making use of recovery techniques, such that it appears as if the crash had never taken place. Another term for crash resilience is **failure transparency**, which is about the system's ability to create the illusion that a failure never happened.

In *Unit 5* we saw that certain architectures can help to prepare the system to cope with failure; for example, by having multiple server machines working together to provide a service. We also looked at the way that various forms of communication handle failure; for example, in the discussion of remote procedure call (RPC) systems with RPC IDs and clocks. In this section we will study what must happen in terms of *data storage* if a system is to behave sensibly when a crash occurs during the execution of a transaction.

Note that methods to make systems more crash resilient do carry some overheads, due to the additional processing or storing that is required. Not every application will strictly require that the user is never aware of a crash happening, whereas in other applications this is a necessity without which the system would be useless.

In this section we will study the concept of stable storage, as well as rollback strategies to deal with crashes in relation to transactions.

6.1 Transactions: failure atomicity and durability

What does it mean if there is a crash in the middle of executing a transaction, and the transaction as a whole has not yet completed? How does the system recover from such a crash?

A first attempt at dealing with such a crash might be to repeat the whole transaction once the system is up and running again. Unfortunately, in many cases this will not be a sensible strategy because the effects of the operations that were executed immediately prior to the crash may already have caused changes to the data.

Example: Buying a trumpet

Take the example of a customer buying a trumpet from an online music shop. The customer has carefully selected a model, they have been told that such a trumpet is in stock, and have entered their credit card details. Imagine that a

crash occurs just *after* they have given their credit card details. It is now unclear to the customer whether they have paid for the trumpet or not. Should they repeat the order?

The trumpet buying scenario shows that a transaction may have several **externally visible effects**. Examples of such externally visible effects are that there may have been communication with another process, or that data has been written to persistent storage.

In the case of the trumpet buying scenario, the facts that the customer has been told that the trumpet is in stock and been asked for their credit card details are externally visible effects. Another example of an externally visible effect is the bank having written to their persistent storage that money should be transferred from the customer's account to the shop's account. Clearly all these externally visible effects are related, and also, clearly, if it is the case that money has been taken out of the customer's account, then this should be clear to the customer.

In this section we leave the issue of communication with other processes, including the messages that are made visible to a human user, aside for a moment, and consider how to deal with persistent data.

6.2 Persistence

We said that for a transaction, if it terminates normally, all its effects are made permanent; whereas if it fails, it is as if nothing had happened. In other words, it should be atomic.

By saying 'nothing has happened' we mean that no externally visible effects must have taken place. This is often referred to as the 'all or nothing' feature of transactions.

If an operation is only half done, the system must be able to make any changes that were made, appear as if they never took place. Thus, a trumpet buying customer would not have any money taken out of their account if the transaction as a whole had not succeeded.

One way of talking about transactions is to say that if they are successful they commit, and if they fail they abort. By committing a transaction we understand that the effects of the transaction are made persistent, because they are written to a persistent store, and usually the store is not merely persistent but also stable. When a transaction aborts, all the effects are made undone, and it is as if the transaction never took place.

Stable storage is the type of storage for which even if there were a fault with the disk on which the data was written the data would persist. This is because stable storage involves storing the data in at least two independent places, so that if one place gets affected by something like a disk crash, the other place is not affected. A number of disk-organisation techniques, collectively known as **redundant array of independent disks (RAID)**, have been proposed. Having a large number of independent disks not only offers opportunities for stable storage, but also opportunities for improved performance. For example, the rate at which data can be read or written may be improved (if these operations can be carried out in parallel).

6.3 Recovery

So, how can we undo parts of a transaction and make it appear as if the transaction never took place? That is the question we address in this subsection.

In order to be able to implement transactions that are failure atomic, it must be possible to go back to the state the system was in before the start of the operation. This is called **rolling back**, and means that the persistent storage will be as it was before the operation started.

We assume here that by a crash we mean that there is a system failure leading to the loss of volatile memory. Non-volatile memory is left intact, but a transaction may have been partly through updating data in persistent storage, and hence the persistent storage may be in an inconsistent state. It is not good enough if information that will help the system recover from a crash is kept in memory. If we want the system to be reliable in the face of a crash, the recovery information must itself be kept secure.

A number of approaches may be used to support rollback, of which we discuss two: **logging** and **shadowing**. Note that logging is the one that is most commonly used.

Logging

When we make a change to a data value, we can physically overwrite the value in persistent storage. If we do that, and it turns out the transaction as a whole does not commit, there may be a problem if we do not have the recovery information at hand to bring the system back to the old state. That is why changes are recorded in a **log**, where the log itself is kept in stable storage.

For each update, a record is kept of the identifier for the transaction, the value of the variable before the change was made (the **before image**) and the value of the variable after the update (the **after image**), as well as a note on whether the transaction (if it completed) committed or had to be aborted.

When the system starts up again after a crash, the log is analysed. Any transaction that was only partially done can be undone. This involves resetting the persistent store to the values before the start of the transaction, as recorded in the log.

Note that the log has to be a **write-ahead log**. That is, we must first write the information to the log, before making the actual change. This is to avoid the possibility of a crash happening at the precise moment that the value is changed in persistent store, but before the record was entered into the log!

Shadowing

For shadowing we make use of duplicate disk pages in stable storage. One page is used to make the updates, and the other page, referred to as the shadow page, is left untouched. Thus every time an update is made, a new value is written to the new page, while the shadow page remains intact.

A page is a unit of storage on a disk.

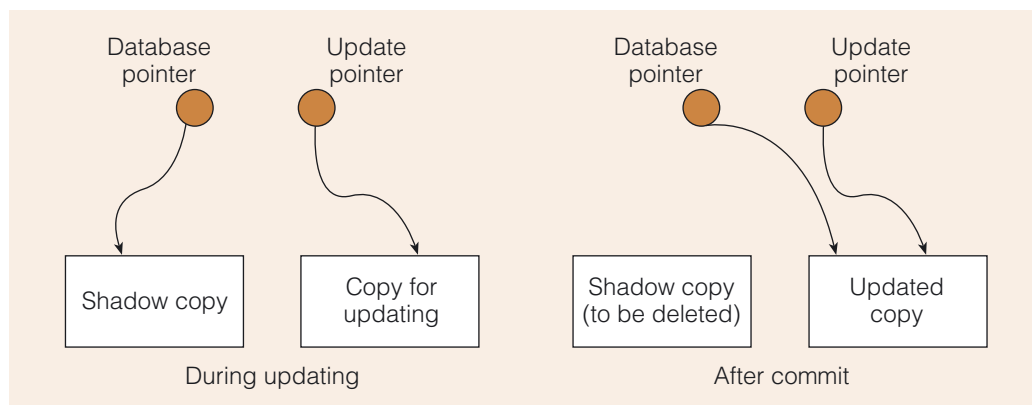


Figure 13 Shadow technique

When the transaction is committed, the new page replaces the old page. It is important that the swapping over of the old for the new page is done in one atomic 'machine-level'

step. This is to avoid the possibility of a crash happening just when the swapping over is taking place. In practice it will involve setting a pointer from one address to another, which is a single machine-level operation.

SAQ 4

- (a) Distinguish between volatile memory, persistent storage and stable storage.
- (b) For crash resilience, what must be done when a transaction aborts?
- (c) Which of the ACID properties of transactions are addressed through the provision of crash resilience?
- (d) What is meant by rolling back?
- (e) In logging, what might happen if log information is written to the persistent store after the data updated by the atomic operation is written there?
- (f) What is shadowing? What essential feature is required to ensure that shadowing is implemented successfully?

ANSWER.....

- (a) Volatile memory holds data that would be lost if the device is switched off (see Section 2), whereas data stored in persistent storage remains even when the device is switched off, or the program that created the data stops executing. Stable storage is even more secure, because data is stored in several places. Therefore, if a disk fails in one location, usually the data survives in a different location.
- (b) For a transaction, when a crash occurs, it must be as though the transaction had never been invoked. Therefore all the effects of an aborted transaction must be undone before it can be restarted.
- (c) (Failure) atomicity and durability. Transactions have to be all or nothing, i.e. failure atomic, and therefore crash-resilience mechanisms are put in place to ensure that a half-completed transaction can be undone should a crash occur. A crash-resilience mechanism also ensures that, if the transaction had committed, its results are definitely stored in persistent store, i.e. durability.
- (d) Rolling back means restoring the persistent store to the state that existed before the start of a transaction that was aborted due to a system crash.
- (e) In logging, the information in the log is used to roll back the persistent store to its state at the start of the transaction in which the crash occurred, by using the old values recorded in the log. If a crash occurs after the data updated by the transaction is written to the permanent store but before the log information is written there (i.e. it is not a write-ahead log), then there will be no record of the original state to facilitate rollback.
- (f) Shadowing is where the results of a transaction are built up in a structure that mirrors part of the persistent store but the persistent store is not updated until the transaction commits. Once the transaction has committed, the updated copy replaces the relevant part of the persistent store.

The essential feature of shadowing is that the replacement of the relevant part of the persistent store by the updated copy should occur in a single operation (e.g. by setting a single pointer). This avoids problems that would occur if a crash happened during the replacement.

7

Communication: JDBC

In this section we study the way in which an application would actually communicate with the database in order to get data in and out of the database.

We will start by looking at the functionality of the database management system (DBMS), and in particular its facility for bridging between Java applications and different types of database. We then go on to study the `Connection` class, which provides methods to make connections with a database and for specifying transactions.

7.1 DBMS functionality and the relational model

In Section 2 we explained the role of the DBMS as being responsible for the management of the database. A DBMS can be seen as an extremely complex set of software programs that controls the organisation, storage and retrieval of data in a database.

There are a number of different DBMSs available, both from commercial vendors and offered free as part of open source projects. The DBMSs all differ somewhat from each other. However, all will have similar core features which will include the following.

- ▶ A modelling language to define how the data should be modelled. For example, the data may be modelled in a hierarchical, relational or networked way, with relational, as described below, being by far the most commonly used structure.
- ▶ Data structures that are optimised to deal with physically storing large amounts of data on a permanent data storage device.
- ▶ A query language to allow users to interactively interrogate the database, analyse its data and update it.
- ▶ A transaction processing facility that would guarantee the ACID properties of transactions.

The most commonly used databases are so-called **relational databases**. The main feature of a relational database is that the data is conceptually organised in the form of a table, also known as a *relation*. Below we show an example of a table with data for a shop selling musical instruments.

Table 22 `INSTRUMENT_MODELS`, a table containing data relating to different models of musical instruments

ID (text)	Category (text)	Name (text)	Description (text)
M48-128	guitar	Bishop Edge	Bass guitar
M66-245	piano	Muzuki AG56	Upright mahogany
M70-362	guitar	Yamala	Standard acoustic
M84-101	trumpet	Amigo 127	Student B-flat trumpet Lacquer finish

Table 22 shows a logical grouping of the data, such that each column contains data of a specific type. The example only shows items of type text, but entries can be of a variety of data types, such as integer, double and more complex types such as date or time. Each row in the table, often referred to as a **record**, holds the related data for one instrument model. We must be aware that ‘under the hood’ the data may be stored in files and various data structures, but we don’t need to know about such details. The table as a whole, with its columns and rows, expresses the **relation** between the various data items.

Requests for information from a database are made in the form of queries, which are stylised questions. For example, the query

```
select * from INSTRUMENT_MODELS where category = 'guitar'
```

requests all records for which the `category` field is ‘guitar’. If this query was executed on the above table, the result would be the records for the guitar models as identified by the codes M48-128 and M70-362.

The set of rules for constructing queries is known as a **query language**. Different DBMSs support different query languages, although **Structured Query Language (SQL)** has become by far the most commonly used. SQL has been standardised so that a program containing SQL statements can communicate with most database systems without having to change the SQL commands.

Table 23 shows some of the SQL statements that are relevant for us.

Table 23 SQL statements

SQL statement	Description of use
create	To create a table (among other things).
insert	To insert a record into a table.
drop	To remove a table from a database.
delete	To remove records from a table.
update	To change values in existing records.
select	To query a database. Returns a <code>ResultSet</code> , which will hold the data extracted from the database.

In 1986 the first ANSI standard for SQL came out. In 1992 there was a major revision of the standard, while in 1999 and 2003 a number of further updates were made.

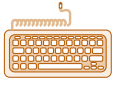
You will see examples of how to use most of these methods in the activity.

7.2 Setting up a connection

When we write a client application, say in Java, to access a database running on a database server, we must first *connect* to the database before sending any SQL commands. In this subsection we give some background to some of the steps involved in setting up such a connection with a database.

Open Database Connectivity and JDBC

The problem in writing applications that need to access databases is that there are many different databases around, and each database vendor has come up with a different interface for making such connections, as well as possibly giving different extensions to the standard SQL. If we are interested in writing applications that can run on a number of different databases, and not limiting our application to just one specific database, we need to make sure that we build in the option of using different interfaces.



Activity 6.1

Creating a database and connecting through JDBC.

- 2 The Java application then interacts with the JDBC driver manager to request a connection to the database with a specified name. As a result of this request, the driver manager then calls every JDBC driver that has been registered with the JDBC driver manager, to see whether the supplied name fits their database. If step 1 has been carried out correctly, the registered driver then responds to the JDBC driver manager.
- 3 The JDBC driver manager is now in a position to send a `Connection` object to the application program. Actually, the object sent is an object implementing the `Connection` interface, where the actual implementation has been provided by the driver. At that point the lines for communication between the application and the database have been established.
- 4 The application program now uses the `Connection` object to communicate with the database.

All the above may seem unnecessarily complicated. Why doesn't the application communicate directly with the driver? Well, it could do that. However, then there wouldn't be a single standard library any more: there are some 50 different protocols and conventions for talking to 50 different databases. The point of JDBC is to avoid that.

Van der Linden summarises the above points as follows.

... your application program knows which database it wants to talk to, and hence which database driver it needs to load. The JDBC driver manager knows how to establish the JDBC end of a database connection, and the driver knows how to establish the database end. They do it. The driver manager gives you back a connection into which you can pour standard SQL queries and get results.

(van der Linden, 2004, p. 618)

In Subsection 7.4 we see how to 'pour' SQL queries into the connection.

7.3 DBMS transaction support

Before we continue, we must explain to what extent a DBMS can offer support for transactions and take charge of concurrency control. For this purpose we show the following two scenarios for a banking system, where a transfer transaction from one bank account to another is put to the system.

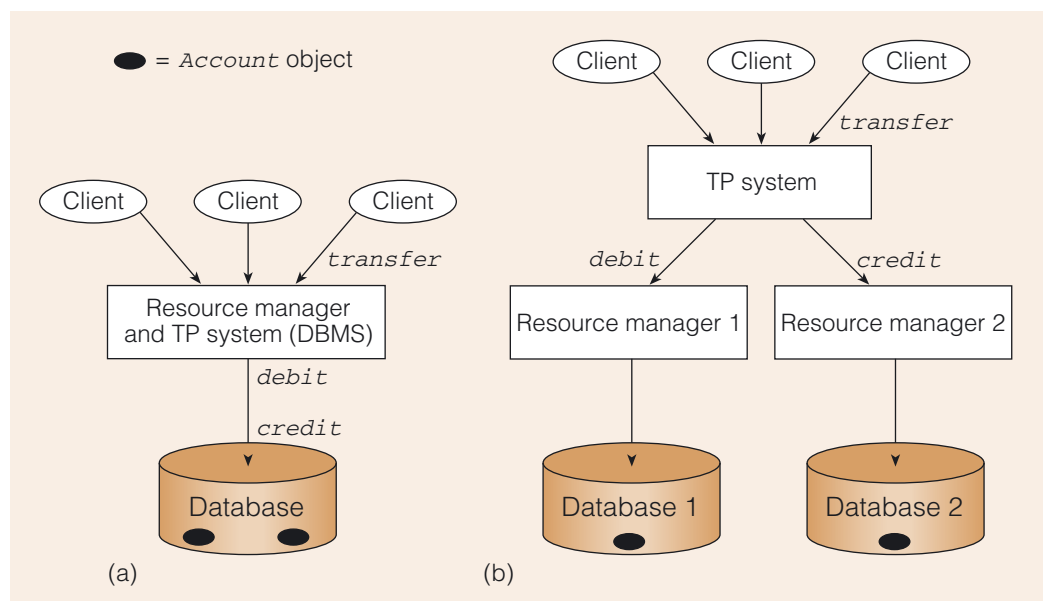


Figure 15 DBMS support for transactions

A DBMS can handle scenario (a): a centralised system, in which both the resource manager and the TP system form an integrated part of the DBMS. Hence the DBMS is able to manage the resources *and* transactions of the local database.

Scenario (b) shows a transaction that relates to data objects residing on different databases (an example of a distributed transaction, which will be discussed in the next section). Since the data is held on different databases, more than one resource manager is required. Also, the TP system needs to be separate from the resource manager, as it is required to coordinate the efforts of a number of resource managers.

A DBMS provides transaction support at the level required for scenario (a) in Figure 15, and we can use JDBC and the `Connection` interface. When the databases and transactions are distributed, we need to use the JTA. We saw an example of how to specify a transaction using JTA in Subsection 3.3.

Note that JDBC is a relatively low-level mechanism for the interaction between a Java application and a database. In *Unit 7* we discuss the way that the communication is established in a Java EE application. This will involve using Enterprise JavaBeans (EJB), which is a higher-level mechanism.

7.4 The `Connection` interface and SQL

The JDBC driver manager returns an object to the application, as illustrated in step 3 of Figure 14. This object implements the `Connection` interface, where the driver will have done the actual implementing. This `Connection` interface is part of the Java SQL library, and here we discuss some of the methods that will enable us to control and execute transactions.

The `commit`, `rollback`, `insert` and `execute` methods

Transaction control is performed by the `Connection` object. When a connection is created, by default it is in the auto-commit mode. This means that each individual SQL statement is treated as a transaction by itself, and will be committed as soon as its execution is finished.

If we want to be able to control more precisely what we consider to be the start of a transaction and what we consider to be its end, we can turn off the auto-commit mode with:

```
connection.setAutoCommit(false);
```

where `connection` is of type `Connection`, and turn it on again with:

```
connection.setAutoCommit(true);
```

Once auto-commit is off, no SQL statements will be committed (i.e. the database will not be permanently updated) until we have explicitly told it to commit by invoking the `commit` method:

```
connection.commit();
```

Another way of looking at this is to say that the `commit` method marks the end of the transaction.

If during the execution of the transaction an exception is raised, then part of the recovery from that involves rolling back the transaction. This is accomplished using

```
connection.rollback();
```

We have put some of these ideas together in a simple scenario. We first show the Java code, and then discuss the steps.


```

try
{
    connection.setAutoCommit(false); //1
    Statement statement = connection.createStatement(); //2

    statement.execute("insert into INSTRUMENT_MODELS values " +
        "('M48-138', 'guitar', 'Hipson Wild', " +
        "'Electric guitar. Solid mahogany')"); //3
    statement.execute("insert into INSTRUMENT_MODELS values " +
        "('M60-445', 'guitar', 'Ibanez', 'Electro classical')"); //4
    statement.execute("insert into INSTRUMENT_MODELS values " +
        "('M66-276', 'piano', 'Bechstein', " +
        "'Baby grand. Black')"); //5

    connection.commit(); //6
    connection.setAutoCommit(true); //7

}
catch(SQLException sqle) //8
{
    System.err.println("SQLException: " + sqle.getMessage()); //9
    connection.rollback(); //10
    connection.setAutoCommit(true); //11
}

```

The main idea of this scenario is that we want the insertion of three new data items into the existing database shown in Table 22 (two guitars and one piano) to be treated as one single transaction. (Note that at this point it is not relevant to delve into the exact reasons as to why these three statements should be considered a transaction.) If all three statements have been inserted successfully, we can commit the transaction; if there is a problem with the insertion of one of these data items, causing an `SQLException`, we need to roll back the transaction.

To better understand the scenario and the effects of the various methods, we go through the statements line by line.

//1 First we switch off auto-commit, thereby indicating that the statements that are to follow need to be considered as a single transaction.

//2 We then create a statement, which will allow us to execute SQL queries.

//3 With this statement we attempt to insert into the `INSTRUMENT_MODELS` table a guitar, as specified by a number of values. Here the `INSTRUMENT_MODELS` table is used as shown in Table 22, with the fields ID, Category, Name and Description.

We are using the SQL statement `insert` here, which allows us to insert a record into a table (see Table 23 for the SQL commands). This `insert` command is then wrapped into an `execute` statement, which is the JDBC way of invoking an SQL statement.

Note that the change to the database has not been made final (committed) yet.

//4 and //5 are as for //3: to insert one more guitar and a piano.

//6 By calling `commit` we mark the end of the transaction, and request the changes made to the database to be made permanent.

//7 The application resets the connection to the default of auto-commit again.

//8 If any of the above statements (in the `try` block) encountered a problem, thus throwing an `SQLException`, they get caught here.


```
//9 Here we are dealing with the actions that need to be taken when an exception is
    raised. First, the exception needs to be reported, in this case the System.err
    stream is being used for error reporting.

//10 When we invoke rollback, we cancel the possible effects of the insertions that
    were carried out in the try block. Note that we do not know exactly where the
    exception occurred, and because we want to avoid the situation where only the first
    guitar is inserted, for example, and not the second guitar and the piano, we undo
    all the work that was done since the auto-commit was switched off. At this point the
    INSTRUMENT_MODELS table is as it was before the insertion.

//11 As in statement //7, we switch back to auto-commit mode, the default setting.
```

Using `select` and `ResultSet`

Here we show an example of using the `select` statement in conjunction with `execute` as follows.

```
statement.execute("select ID, Name, " +
                  "Description from INSTRUMENT_MODELS " +
                  "where category = 'guitar'");
ResultSet results = statement.getResultSet();

System.out.println("Guitar models: ");
while (results.next())
{
    String IDString = results.getString("ID");
    String descriptiveString = results.getString("Description");
    String nameString = results.getString("Name");
    System.out.println(IDString + " = " + nameString + ": " +
                      descriptiveString);
}
```

The output of running the above query (with its additional processing) will be:

```
Guitar models:
M48-128 = Bishop Edge: Bass guitar
M70-362 = Yamala: Standard acoustic
M48-138 = Hipson Wild: Electric guitar. Solid mahogany
M60-445 = Ibonez: Electro classical
```

In the query we had requested only the ID, Name and Description of all the instrument models in the guitar category. The `ResultSet` object returned by the query is essentially a two-dimensional array. The `ResultSet` object can be processed as shown, in order to obtain its contents and display it in our preferred format.



Activity 6.2

Updating and querying a database.

7.5 JDBC – transaction isolation levels

JDBC recognises a number of **transaction isolation levels**, which can be of interest if we want to fine-tune the concurrency levels for the transactions. The idea behind these isolation levels is that there may be applications for which it is not problematic if the very strict definition of serialisability is not fully adhered to. For example, in some applications the priority may be to be able to read information at the moment it is requested, rather than waiting because data is in the process of being updated, even if this means that the data is slightly out of date, or has not been fully confirmed as committed.

The `Connection` interface enables the developer to set different transaction isolation levels for use in JDBC, based on the types of read allowed on the data. Three types of read are specified.

- ▶ A **dirty read** occurs if transaction T_1 updates a value and transaction T_2 reads this value before transaction T_1 commits.
- ▶ A **non-repeatable read** occurs if a transaction, say T_1 , reads a data item, and another transaction, say T_2 , then modifies or deletes that data item and commits. If T_1 then rereads the data item, it will either receive a modified value or discover that the data item has been deleted.
- ▶ A **phantom read** occurs if a transaction T_1 reads a set of data items satisfying some condition, say 'select all instruments with a price < 250', and transaction T_2 then creates a data item that satisfies the search condition of T_1 and commits. If T_1 then repeats its read with the same condition, it gets a set of data items that is different from the first time.

Table 24 shows the five isolation levels that can be set for a `Connection` object. The default is that the transaction is serialisable, which is how we have defined transactions throughout this unit.

Table 24 Transaction isolation levels

Transaction isolation level	Description	
TRANSACTION_NONE	Indicates that transactions are not supported.	Low isolation
TRANSACTION_READ_UNCOMMITTED	Dirty reads, non-repeatable reads and phantom reads can occur.	
TRANSACTION_READ_COMMITTED	Dirty reads are prevented; non-repeatable reads and phantom reads can occur.	
TRANSACTION_REPEATABLE_READ	Dirty reads and non-repeatable reads are prevented; phantom reads can occur.	
TRANSACTION_SERIALIZABLE	Dirty reads, non-repeatable reads and phantom reads are prevented.	High isolation

In Table 24 the isolation levels are shown in ascending order of strictness. Typically, the higher the level of isolation, the slower the application executes, due to increased overheads and decreased concurrency between users. The developer must balance the need for performance with the need for data consistency when making a decision about what isolation level to use. Of course, the level that can actually be supported depends on the capabilities of the underlying DBMS.

When a new `Connection` object is created, its transaction isolation level depends on the driver, but normally it is the default for the underlying data source. A user may call the method `getIsolationLevel` to find out what the level is, and `setIsolationLevel` to change the transaction isolation level. The new level will be in effect for the rest of the connection session. To change the transaction isolation level for just one transaction, a user needs to set it before executing any statements in the

transaction and then it can be reset after the transaction terminates. However, changing the transaction isolation level *during* a transaction is not recommended as it will trigger an immediate call to the method `commit`, causing any changes up to that point to be made permanent.

SAQ 5

- (a) What does the term *relation* refer to in relational database models?
- (b) What are the core features of a DBMS?
- (c) What is the purpose of JDBC?
- (d) What are the steps involved in setting up a connection between a Java application and a database, when using JDBC?

ANSWER.....

- (a) Relation refers to the logical grouping of the data, which is often represented as a table, with rows and columns. Each column contains data of a certain type, and each row, also known as a record, contains the details for an object in the table. The relation is the table as a whole.
- (b) The core features of a DBMS are:
 - ▶ a modelling language to model the data;
 - ▶ data structures for storage of data;
 - ▶ a query language to retrieve and update data;
 - ▶ a transaction concurrency control mechanism.
- (c) The purpose of JDBC is to provide a standard interface for communication with a large number of different databases.
- (d) The steps involved in setting up the communication are:
 - 1 the application causes the database driver to be loaded;
 - 2 the application requests a connection for a particular database, and the driver manager requests all registered drivers to respond with a connection;
 - 3 the driver manager returns a connection to the application;
 - 4 communication through the JDBC connection to the database.

SAQ 6

- (a) Why is a DBMS not able to offer concurrency control for distributed transactions?
- (b) What is the default setting for transaction processing when executing SQL statements?
- (c) How can the default setting be altered?
- (d) Explain how we can deal with possible `SQLExceptions` occurring during the execution of SQL statements arriving from a Java application.

ANSWER.....

- (a) Each DBMS has one resource manager. A distributed transaction involves objects residing in several locations held in several databases, and will require more than one resource manager to manage the concurrency control.
- (b) The default setting of the `Connection` object is that each SQL statement is treated as a transaction.

- (c) The default setting can be altered with the command `setAutoCommit(false)`, in which case all the statements after that are treated as forming part of one transaction, which will finish as soon as the `commit` or the `rollback` command is executed.
- (d) The SQL statements that form one transaction should be grouped in a `try-catch` clause. If an exception occurs during one of the statements being executed, this can be caught, and the transaction can then be rolled back using the `rollback` method.

SAQ 7

- (a) What is the point of having transaction isolation levels?
- (b) Explain the different forms of read: dirty read, non-repeatable read and phantom read.

ANSWER.....

- (a) For some applications it may be appropriate to fine-tune the concurrency control, based on allowing certain types of read to happen.
- (b) A dirty read occurs if a transaction reads values that are written by another transaction that hasn't committed yet.

A non-repeatable read occurs if a transaction reads the same object twice during execution and finds a different value the second time, although the transaction has not changed the value in the meantime.

A phantom read occurs when a transaction re-executes a query, returning a set of data that satisfies the condition in the query, and finds that the set of data has changed as a result of another recently committed transaction.

8

Distributed databases

Data belonging to one company or organisation is increasingly distributed over several databases, rather than being held in a single, large, centralised database. The software that manages such databases is called a distributed DBMS. The parts of such a distributed DBMS have a large degree of mutual independence. One of the major challenges for the development of such systems is how to make it appear to the user that they are dealing with just one database.

Each part of the distributed DBMS will participate in the execution of transactions which may either access data stored at that particular site, or may access data held at several sites. When data is distributed in several places it is more complicated to query and to update the data.

In this section we briefly outline some of the major design issues for distributed DBMSs, including how to deal with distributed transactions.

8.1 Fragmentation and transparency

Large companies may operate in different parts of the world, with projects running in different localities, and groups of employees being employed at different offices. Under such circumstances it may be beneficial if data relating to the staff of each office is held locally, whereas it must also be possible to query the database of the company as a whole. This is what is known as **fragmentation**, because the data of the company as a whole has been fragmented into different parts, held in different physical areas of storage.

It may also be the case that databases that are initially separate need to be joined together; this may arise when, for example, one company takes over another company or where a company decides to offer additional services through another company. In such instances the separate databases can be put together to form one distributed DBMS (although some work may need to be carried out to ensure that it is technically possible to merge the databases).

An important design issue in distributed DBMSs is whether and how best to fragment the data. Given that data is held in tables, there are two main alternatives: slicing a table horizontally or slicing it vertically.

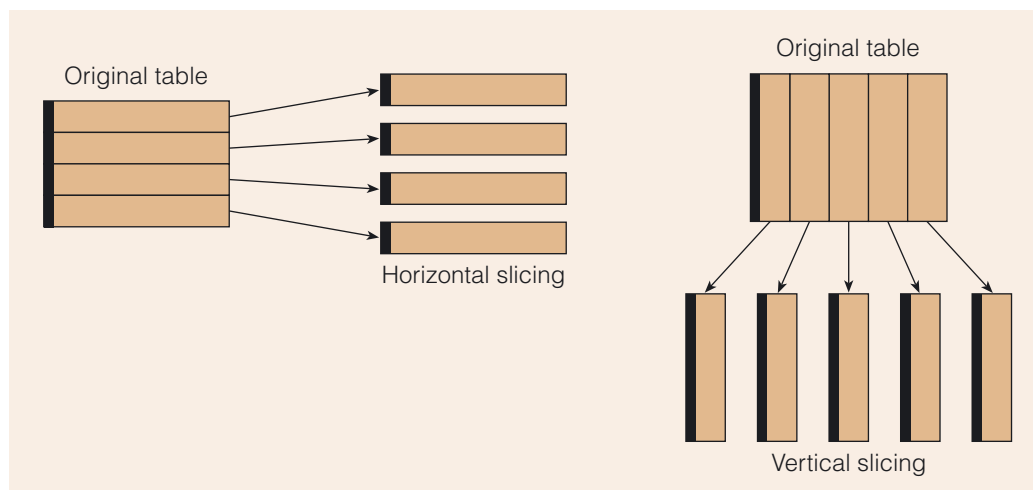


Figure 16 Horizontal and vertical slicing

With horizontal slicing, we would end up with a set of rows that may have a certain field in the record in common – for example, all the records that relate to customers in Finland are stored on the Finnish fragment and are kept separate from all the records relating to Canada. The effect of vertical slicing is that, for example, all personnel data is at one physical location, all financial data at another, and so on.

Whichever form of slicing is used, it must be possible to put the slices back together again. To this end each slice holds the primary key (in Figure 16 shown as the shaded areas) which is a piece of data that uniquely identifies each record in the database. We saw an example of this in Table 22, where the primary key is ID. When using horizontal slicing each record includes the primary key, so by searching on the primary key the whole record can be obtained. In vertical slicing, the primary keys are duplicated for each slice, and a complete record for a given primary key can be obtained by matching identical primary keys in the various slices.

Even though the database is fragmented, the distributed DBMS would aim to provide facilities for the transparent management of the database. Here we mean that to the users of the database, the data is presented in a uniform way, and they would not be aware that some parts are held elsewhere.

8.2 Data replication

Another major design decision in distributed DBMSs concerns **data replication**, by which we mean that multiple copies of the same data are held at different places in the system, in contrast to fragmentation which is about disjoint subsets of data. This is shown in Figure 17.

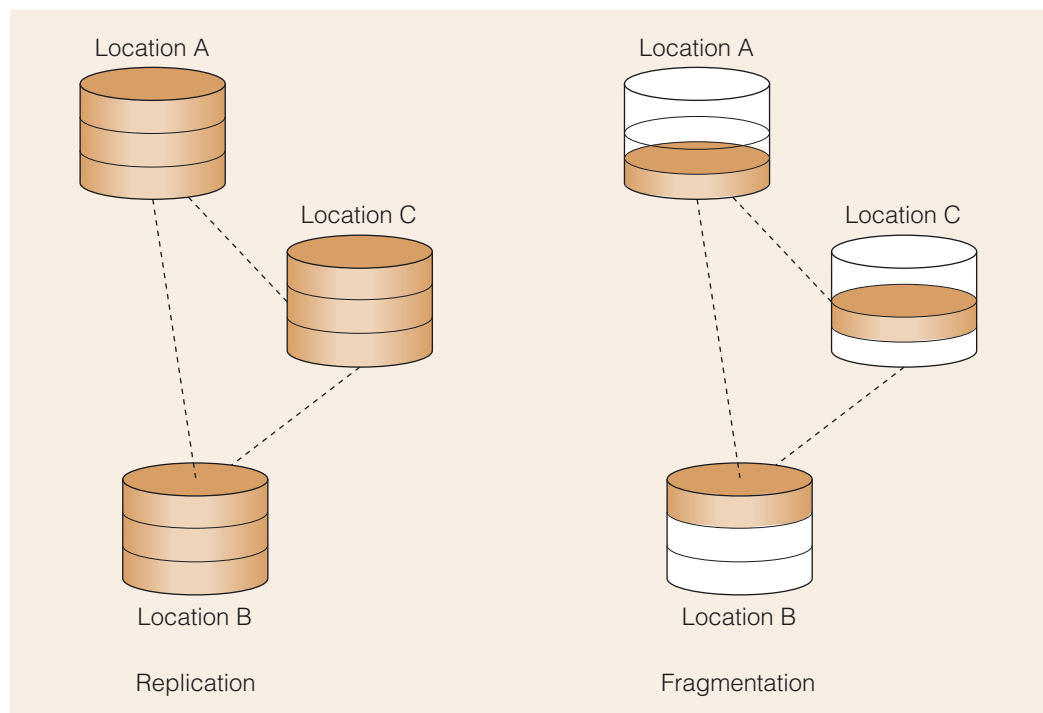


Figure 17 Replication versus fragmentation

The replication of data can improve performance when the data is 'closer' to where the processing of the data takes place. Furthermore, if part of a system fails for some reason, the replication means that data is still held securely. However, replication of data also poses major problems, particularly if the nature of the data is such that it is continuously updated, rather than merely queried; updating all the replicas may be a costly affair.

Again the idea of transparency is important – should users be aware that there are multiple copies of the data, or should the data be managed in such a way that to the user it is as if only one copy exists? The user probably does not want to be lumbered with details of such replicas. However, when the responsibility of specifying that an action needs to be executed on multiple copies can be delegated to the user application, it makes the management of the transactions simpler for the distributed DBMS. Here we discuss two distributed DBMS strategies for the replication of data.

- ▶ **Synchronous replication.** Updates to all the replicas of a data item are seen as part of a transaction. If a transaction performs an update on a data item that has replicas, then all the updates must be made before the transaction can be committed. One problem with this approach is that if one of the sites holding a replica is unavailable at the time of the transaction, the transaction cannot complete. This form of updating carries significant overheads as it requires a large amount of communication to coordinate the synchronisation.
- ▶ **Asynchronous replication.** Here the replicated part is updated *after* the source database has been modified. This is a much simpler approach, but it has the disadvantage that it may take some time for all parts of the database to regain consistency.

Furthermore, if there are several copies of data, there are issues to do with the *ownership* of the data. Which site has the privilege for keeping data up to date? There are three main types of ownership.

- 1 **Master–slave ownership (or asymmetric replication).** The asynchronously replicated data is owned by one site, the master site, and only this site is permitted to make the updates. In the previous unit we discussed the publish/subscribe model, which would be used in this form of replication. The master site would act as the publisher, and other sites can show their interest in any changes to data by subscribing. The subscribed sites could receive ‘read only’ copies, but would not be allowed to make changes to the master themselves.

The master–slave form of replication may be a suitable way of organising responsibilities when dealing with a mobile workforce. For example, consider a company employing telephone engineers who travel from site to site to attend to engineering activities. The data of each site would be held by the master site, and the engineers can download the appropriate local data for the site they work on. When they have finished their job, they can request that the data be updated with their local version, but the privilege of making the update remains with the master site.

- 2 **Workflow ownership.** This form of ownership is more dynamic, as the right to make updates moves from site to site. However, at any one moment, there is only ever one site that may update that particular data set.

This may be a more suitable form of update ownership for order processing systems which follow a series of steps, such as order entry, credit approval, invoicing, shipping, and so on, with different sites taking responsibility for the different steps.

- 3 **Update anywhere ownership.** This form of ownership creates essentially a peer-to-peer (P2P) environment where multiple sites have equal rights to update replicated data. The local sites are allowed to function autonomously. Shared ownership can lead to conflict scenarios and it would have to employ a methodology for conflict detection and resolution.

This form of ownership is often employed in open-source projects, where it is referred to as concurrent versioning systems (CVS). An open source project will consist of a number of files, and anyone who wants to work on the project can do so, and submit their updated version of a file. Those working on the projects are encouraged to submit their updates very regularly; otherwise they could find that the

file they are working on may have been changed since they downloaded it, and that their changes will now not be accepted by the CVS.

8.3 Distributed transactions

The access to the data of a distributed system is typically accomplished through transactions, which must preserve the ACID properties. We need to consider two different types of transaction:

- ▶ **local transactions**, which access and update data in a single local database;
- ▶ **global transactions**, which access and update data in several local databases.

As an example of such a global transaction, we can think of our favourite – the transaction to transfer money between bank accounts, which may well involve accounts held by several different banks, each with their own database. In Figure 15(b) we briefly referred to this example, where the debit and the credit part had to be executed on different databases.

In Subsection 3.5 we explained that the processing of transactions is managed by a transaction processing system, which takes care of all locking and recovery issues. The same risk of conflicts exists when dealing with global transactions in Figure 15(b), where we would need several transaction processing systems, one for each location, to deal with conflicts and enable recovery if necessary.

A distributed transaction processing system maintains the ACID properties in distributed transactions through a **commit protocol**. A commit protocol allows multiple processes to coordinate the committing or aborting of a transaction in an atomic way. The most common commit protocol is the two-phase commit protocol described below.

Two-phase commit protocol

The **two-phase commit (2PC)** protocol, as the name implies, involves two phases: a **prepare phase** and a **resolution phase**. For each transaction, one process acts as the **coordinator**. The coordinator oversees the activities of the other participants (each of which deals with only a part of the transaction) in the transaction to ensure a consistent outcome.

In the prepare phase, the coordinator sends a message to each process in the transaction, asking each process to prepare to commit. When a process prepares, it guarantees that it can commit the transaction, after which it can no longer decide to abort. If a process cannot prepare (i.e. if it cannot guarantee that it can commit the transaction) it must abort. In the resolution phase, the coordinator tallies the responses. If all participants are prepared to commit, the transaction commits; otherwise, it aborts.

In the 2PC protocol, even if only one process decides to abort, the transaction as a whole has to abort. The 2PC protocol is, in fact, a kind of voting algorithm, where all participants can vote to go ahead with the transaction, but each participant also has the power to veto. Here we walk through the steps that would take place in the 2PC protocol.

- 1 The coordinator writes a *beginCommit* record in its log and sends a message to each participant asking them to *prepare*.
- 2 When a participant receives a *prepare* message, it checks if it is able to commit. If this is the case, it sends a *votingCommit* message to the coordinator; otherwise it sends a *votingAbort* message.
- 3 If the participant has decided to vote for abort, it can now forget about the transaction, because an abort decision serves as a veto.
- 4 When the coordinator has received a reply from every participant, it decides whether to commit or to abort the transaction. Even if only one participant has voted

Not to be confused with two-phase locking.

negatively, the coordinator has to decide to abort, and sends a *globalAbort* message to all participants. Otherwise a *globalCommit* message is sent to all participants.

- 5 The participants either commit or abort, depending on the message received, and send an acknowledgement.
- 6 When all the acknowledgements have been received, the coordinator terminates the transaction by writing an end-of-transaction record in the log.

The protocol responds in different ways to various types of failure, which include failure of the participating site, failure of the coordinator, or a network failure. If a site fails before it has been able to send a response, the coordinator assumes that the site had responded with an abort message. On the other hand, if the site fails after sending a message to confirm that it will commit, the coordinator will continue on the basis of confirmation. It is then up to the site that failed to work out what to do when it recovers from the failure. A coordinator failing midway through sorting out a transaction is more problematic, as the participating sites may not be able to decide whether to commit or abort, and become blocked, waiting for the recovery of the coordinator.

Note that if a distributed transaction involves objects that are replicated, the algorithm needs to take this into account. We will not further discuss this.

8.4 Relationships between problems

The study of distributed databases is a vast area of research in its own right, and we have only touched on a couple of the issues that the designer of such a system would have to address.

Other areas of concern are the design of algorithms to deal with queries that relate to distributed data, the structuring of directories to hold the information about where all the fragments of the database are held, the management of distributed deadlock and the reliability issues of the distributed DBMS, to mention but a few.

Many of the issues are connected with each other. The issues of fragmentation and replication, for example, cannot be addressed independently of the other concerns of distributed DBMSs. When fragments are replicated and distributed, this has an effect on the concurrency control strategies that might be employed.

We have seen that Java offers very useful facilities for the access to databases, through the JDBC and `Connection` interface, and this, with the combined power of the concurrency mechanisms of Java, makes it a very powerful language that continues to be used in large distributed applications.

Exercise 8

A distance teaching university has a number of databases to store data relating to its staff, students and courses. The university is managed in several regions, where each region can enrol students and employ staff. A central site is responsible for the management of the courses, including the delivery of course materials to students.

The database for the data of the university's staff stores their personal details: their staff ID, name, home address, postcode, position, and the code for the region. The responsibility for keeping the staff details up to date lies with the region.

Table 25 University staff details

ID	Name	Address	Postcode	Position	Region
8455	Janssen	21 West Hill, Stevenage	XX6 7XX	Lecturer	4
8562	Loveridge	69 Hollybush Way, Reading	YY3 1Y	Staff tutor	2
1782	Admiral	189 Hill Avenue, Reading	WW5 2YY	Clerical	2
2133	Young	6 Oak Tree Lane, Glasgow	ZZ4 9ZZ	Clerical	9
2744	Davies	7 Old Park Road, Letchworth	TT7 8TT	Course manager	4
2588	Patel	13 Castle Close, Inverness	AA3 2AA	Lecturer	9

Briefly explain the two strategies for the fragmentation of data in a distributed DBMS. In your answer give examples relating to the staff database, and explain which strategy would be most suitable for the university.

Discussion

There are two ways in which data can be fragmented: horizontally and vertically. Whichever way the database is fragmented, horizontally or vertically, the fragments when put together, should form one database for the university as a whole.

Horizontal fragmentation refers to the slicing of a table such that we have only a subset of all the rows of the original table. Within a horizontal slice, there would be a column for which all the rows have a common (or similar) value so that we might end up with slices according to age or salary bands, or postcode groupings.

Vertical fragmentation is where a table is sliced vertically such that we have only a subset of all the columns of the original table. The data is thus divided into groups of one or more columns (e.g. for staff, we might separate the columns with home address details and postcode from the columns relating to salary and pension details).

In this scenario the strategy of horizontal fragmentation is more appropriate. We could have a horizontal slicing so that staff are grouped by region. This means that for each member of staff their entire record is managed by the region. It is difficult to think of a vertical slicing that would be meaningful in this scenario.

SAQ 8

- Give two strategies for the replication of data.
- Differentiate between three forms of ownership for the replication of data.
- What do we mean by transparent management of a distributed DBMS? How does this apply to fragmentation and replication?
- What does the 2PC protocol aim to achieve?

ANSWER.....

- Two strategies for the replication of data are synchronous and asynchronous replication. In synchronous replication all replicas of a data item are updated at the same time, when a data item is being updated. A transaction cannot commit unless all replicas will be updated.

In asynchronous replication the replicas are updated after the source has been updated.

- (b) Three forms of ownership for the replication of data are:
- ▶ master–slave ownership – only the master site is permitted to make updates;
 - ▶ workflow ownership – permission to update the data moves along a certain path;
 - ▶ update anywhere ownership – permission is equally shared between peers.
- (c) Transparent management is an important aim of a distributed DBMS, meaning that to the user of such a system it should appear as if they are dealing with just one database. The management of the distribution involves hiding the low-level details of where everything is stored exactly (fragmentation) and whether there are several copies of each data item (replication).
- (d) The aim of 2PC is that the committing of a distributed transaction is done atomically – that is, either the entire transaction has taken place and its effects are made durable, or it is as if nothing has happened.
-

9

Summary

This unit began by exploring the main functionality of a database in a tiered system and its role in the persistent storage of data.

We looked at transactions as the means of querying and updating data in a database. A transaction is defined as a sequence of operations that belong together, thus introducing the notion of failure atomicity. A state transition diagram for the states of a transaction identified that a transaction can finish either by committing or by aborting. We also looked at the system structure of a transaction processing system to ensure that the ACID properties are adhered to.

We then explored in some detail the concurrent execution of transactions. You learnt that making the execution of transactions serialisable (i.e. as though they execute in some serial order) guarantees the ACID property of atomicity. You saw that serialisability can be guaranteed if the conflicting operations of several transactions are executed in a consistent order. In particular we observed that if a precedence graph, representing a schedule for the execution of the conflicting operations for a number of transactions, contains no cycle, the schedule must be serialisable.

You then went on to study the two mechanisms that can be used to effect a serialisable execution of concurrent transactions: 2PL and TSO. The major problem with 2PL is the possibility of deadlock, which can be overcome by aborting transactions with timed out lock requests. TSO is a simple and efficient mechanism that requires no central control and does not lock objects for longer than the duration of an operation. However, it lacks flexibility and can lead to cascading transaction aborts.

Crash resilience makes it appear to the users of a system that a crash never occurred. We studied two mechanisms for implementing crash resilience: logging and shadowing.

A Java application can communicate directly with a database using the JDBC as an intermediary. The JDBC will assist in making the link with the driver of the database that we need to work with, and creates an object which can be used to execute SQL statements for the querying and updating of data. This form of communication is suitable only if dealing with local transactions, using a single local database.

In the case of global, or distributed, transactions that involve several databases, a more sophisticated process for the committing of transactions, the 2PC protocol, should be used. This protocol uses several TP systems, one for each site, and a coordinator process to oversee the commit protocol.

Fragmentation and replication are aspects that need to be considered when designing distributed database systems. Although these facilities are useful and make the system more robust and more responsive, they also complicate matters. When data is dispersed in different places, more effort has to go into deciding whether we are looking at the most recent version of the data, whether all sites involved can agree on an update, and who is in control of the data. However, given the current state of internet activity and internationalisation, the distributed nature of data is unlikely to disappear, and is more likely to increase over the years to come.

LEARNING OUTCOMES

When you have completed your study of this unit you should be able to do the following:

- ▶ be aware of the role of databases in providing persistent storage of data;
- ▶ explain the concept of transactions, and distinguish between the states a transaction can be in;
- ▶ understand the theory of serialisability for transactions, and use precedence graphs to check the serialisability of a given schedule of operations;
- ▶ explain the ACID properties of transactions, and how atomicity and durability are implemented through the provision of crash resilience, and isolation and consistency through concurrency control methods;
- ▶ define and distinguish between different concurrency control methods for transactions;
- ▶ understand the importance of crash resilience, and differentiate between different techniques to enable rollback;
- ▶ set up and use a connection from a Java application to a database and be able to execute simple SQL statements;
- ▶ be aware of some design issues for distributed DBMSs and explain the workings of the 2PC protocol for distributed transactions.

References

- Gray, J. (1981) 'The transaction concept: virtues and limitations' in *Proceedings of the 7th International Conference on Very Large Databases*, September 1981, Tandem Computers Incorporated. Also available online at <http://research.microsoft.com/~gray/papers/theTransactionConcept.pdf> (Accessed 13 July 2007).
- van der Linden, P. (2004) *Just Java 2* (6th edn), Sun Microsystems Press.

Glossary

abort Deliberately stop something that has started.

Aborted (transaction state) The state a transaction is in after it has been rolled back to undo any partially completed actions of the transaction.

ACID properties The desirable properties of a transaction: atomicity, consistency, isolation and durability.

acquire phase The phase in two-phase locking in which the locks are gradually acquired, and in which no locks can be released.

Active (transaction state) The state a transaction is in as soon as it has started, and in which it remains while executing.

after image In logging, the name given to the value of a variable after an update is made.

asymmetric replication See **master–slave ownership**.

asynchronous replication A strategy for replication of data, in which a replicated part is updated some time after the source database is updated.

before image In logging, the name given to the value of a variable before an update is made.

cache manager (in a transaction processing system) The part of the transaction processing system that organises the objects in main memory and interfaces with the persistent storage.

cascade of aborts The situation where a transaction has had to be aborted, and this in turn leads to many other transactions having to abort.

commit protocol A protocol to commit transactions. See **two-phase commit (2PC) protocol**.

Committed (transaction state) The state a transaction is in after the transaction has completed successfully and all the updates to the data have been made permanent (persistent).

conflicting operations Two operations *A* and *B* are said to be conflicting if, starting from some initial state, the result of executing *A* then *B* is not the same as the result of executing *B* then *A*. Also known as non-commutative operations.

coordinator In the two-phase commit protocol for distributed transactions, the process that oversees the activities of other participating processes, each of which is dealing with only part of a transaction.

crash resilience The extent to which an application is supported in recovering from system crashes.

data replication Holding multiple copies of data items in different locations.

database In computing, a structured collection of records or data that is stored in a computer so that a program can consult it to answer queries.

database driver The implementation details of a particular database are handled by the database driver for that database.

database management system (DBMS) The computer program used to manage and query a database.

database metadata Literally, data about data. For example, data about the vendor of the database system and data about the way the database records are stored.

directed acyclic graph (DAG) In graph theory, a graph with nodes and edges, where each edge has a definite direction (going from one node to another). The graph has no cycles, meaning that there is no path from a node that leads back to itself.

dirty read A dirty read occurs if a transaction reads values that are written by another transaction that has not committed yet.

exclusive lock See **write lock**.

externally visible effect Any communication that an operation may have with another process, user or storage device.

Failed (transaction state) The state a transaction is in when the discovery is made that normal execution cannot proceed.

failure atomicity A form of atomicity, also known as all-or-nothing atomicity, which is one of the properties of a transaction. It refers to the fact that if there is some kind of failure which causes a part of the transaction not to be completed, then the entire transaction should be rolled back.

failure transparency Another term for crash resilience, referring to a system's ability to create the illusion that a failure never happened.

fragmentation Slicing a database into different fragments, such that each part is stored in different physical areas of storage.

global transaction A transaction that accesses and updates data in several databases (contrast with **local transaction**).

granularity of a lock A design consideration for algorithms based on locks, regarding the size of the object that needs to be locked.

intermediate state The state that a system is in while it is in the middle of executing an operation. In transaction processing it is desirable that other concurrent transactions avoid seeing the intermediate state of a given transaction.

Java Transaction API (JTA) A standard Java interface for demarcating transactions.

JDBC An API for the Java programming language that defines how a client may access a database.

local transaction A transaction that accesses and updates data in a single local database (contrast with **global transaction**).

log The place in which logging information is recorded. The log is stored in stable storage.

logging The process of recording the new value and the old value and the name of a transaction so that the persistent store can be rolled back to the previous state. A log will also contain information indicating the start of a new transaction, the number of update records and the fact that the transaction has committed. Log information must be written before the data in permanent store is changed.

lost update problem The situation where, during the execution of a transaction, intermediate results (i.e. results of sub-operations) become accessible to other operations, with the potential danger that a previous update becomes lost.

master–slave ownership Also referred to as asymmetric replication. A style of ownership of data in schemes for replication of data where data is owned by the master site, and only the master site is allowed to make updates to the data.

non-repeatable read A non-repeatable read occurs if a transaction reads the same object twice during execution and finds a different value the second time, although the transaction has not changed the value in the meantime.

Open Database Connectivity (ODBC) An interface to SQL-based databases, which aids the communication with a database and the access to the database metadata. The aim of ODBC is to enable access to a great number of databases, independent of the programming language of the application that seeks access to a database, and independent of the database systems and operating systems.

optimistic algorithm In concurrency control for transactions, an algorithm that assumes that conflicts will occur rarely, and that takes measures after the event. Example: **optimistic concurrency control**.

optimistic concurrency control A concurrency control algorithm (optimistic) that works with the assumption that there is not much concurrency contention between transactions. It allows transactions to proceed without locking or time-stamping, and only just before it commits does it perform a validation check. It has three phases: working phase, validation phase and update phase. See also the definition of each phase.

Partially Committed (transaction state) The state a transaction is in after the final part of the transaction has taken place, but before the transaction has committed.

persistent storage A form of storage of data which outlives the execution of the program that created it, e.g. a database or a file.

pessimistic algorithm In concurrency control for transactions, an algorithm that assumes that there are likely to be many conflicts and that takes measures to avoid them. Examples: **two-phase (2PL) locking** and **time-stamp ordering (TSO)**.

phantom read A phantom read occurs when a transaction re-executes a query, returning a set of data that satisfies the condition in the query, and finds that the set of data has changed as a result of another recently committed transaction.

precedence graph A way of representing the order constraints among a collection of transactions (or other statements). The nodes of the graph represent the transactions, and there is a directed edge from node *A* to node *B* if transaction *A* must be executed before transaction *B*.

prepare phase In the two-phase commit protocol for distributed transactions, the phase in which each process involved with dealing with part of the transaction has to send a message to the coordinator to indicate whether it intends to commit. See also **resolution phase**.

query language The language that is used with a relational database in order to query the database and retrieve information.

read lock Also referred to as a shareable lock. A lock that needs to be taken out if a process needs to read from a data resource. Several processes can hold a read lock.

record In a relational database, a record is a set of values for one entry into a database.

recovery manager (in a transaction processing system) The part of a transaction processing system that deals with the recovery from crashes.

redundant array of independent disks (RAID) A redundant array of independent drives (or disks), also known as a redundant array of inexpensive drives (or disks). RAID is an umbrella term for data storage schemes that divide and/or replicate data among multiple hard drives. They offer, depending on the scheme, increased data reliability and/or I/O performance.

relation In a relational database, a relation is also known as a table. The table has a fixed number of uniquely named columns, and each row represents a record for an entry into the database.

relational database A database model, where the data is conceptually modelled as a relation. It uses a collection of tables to represent both the data and the relationships between the data.

release phase The phase in two-phase locking in which the locks are released, and in which no more locks are acquired.

resolution phase In the two-phase commit protocol for distributed transactions, the phase in which the coordinator decides on whether or not the transaction as a whole can commit or not, based on the responses received from the processes involved in carrying out parts of the transaction.

rolling back Restoring the state of the persistent store to the one that existed before the start of an atomic operation that was aborted due to a system crash.

schedule (for transactions) A sequence of operations that specifies the chronological order in which the operations of concurrent transactions are executed.

scheduler (in a transaction processing system) The part of the transaction processing system that puts together serialisable schedules for the operations of transactions.

serial schedule A schedule for the operations of concurrent transactions, in which the operations of one transaction all occur before the operations of the next transaction, such that the transactions execute in a strictly serial fashion. There is no interleaving.

serialisability A schedule for a group of concurrent transactions is serialisable if, and only if, it produces the same result as if the transactions had executed in some serial order. Serialisability is also referred to as serial equivalence.

shadowing Where the results of a transaction are built up in a structure that mirrors the persistent store but the persistent store is not updated until the transaction commits. Once the transaction has committed, the shadow structure replaces the persistent store.

shareable lock See **read lock**.

stable storage An abstraction which means that the results of an atomic operation are written to two independent places in a persistent store.

strict two-phase locking (strict 2PL) A form of two-phase locking in which all locks are released on commit.

Structured Query Language (SQL) The most widely used and standardised query language for relational databases.

synchronous replication A strategy for replication of data, where all copies are updated at the same time.

time-stamp ordering (TSO) An algorithm for concurrency control for transactions based on time-stamps. A time (such as the time at which the transaction started) is associated with each transaction and is recorded by every object involved with each operation invoked by the transaction. This time-stamp is used to serially order transactions.

topological sorting In graph theory, a topological sort of a directed acyclic graph is a linear ordering of its nodes such that each node comes before all nodes to which it has edges. Every directed acyclic graph has one or more topological sorts.

transaction A sequence of atomic actions, with the property that either the entire sequence is performed, or no part of the sequence is performed at all.

transaction isolation level The levels that allow fine-tuning of transactions, based on different reads: dirty reads, non-repeatable reads and phantom reads.

transaction manager (in a transaction processing system) The part of a transaction processing system that provides an interface between the clients and the system, and which passes on requests for transaction processing to the scheduler.

transaction processing (TP) system A layer of middleware to manage the correct and efficient execution of transactions. Also known as a transaction processing (TP) monitor.

two-phase commit (2PC) protocol A protocol for distributed transactions, where one process acts as the coordinator for several parts of a transaction. All parts of the transaction must agree to commit in order for the transaction to commit as a whole. There are two phases: the prepare phase and a resolution phase.

two-phase locking (2PL) An algorithm for concurrency control for transactions based on locking. Locks are acquired for a transaction as they are needed (acquire phase), and no lock can be released until all locks have been acquired. When locks are released, no more locks can be acquired (release phase).

update anywhere ownership A style of ownership for data in schemes for replication of data, where anyone is allowed to update. Often used in the peer-to-peer environment of open source projects.

update phase (in optimistic concurrency control) The phase in which a transaction has been validated, and can now commit.

validation phase (in optimistic concurrency control) The phase in which a transaction gets validated. If validation succeeds, the transaction can commit.

workflow ownership A style of ownership of data in schemes for replication of data, where the right to make updates to replicated parts moves from site to site.

working phase (in optimistic concurrency control) The phase in which a transaction works with a tentative version of the objects.

write lock Also referred to as an exclusive lock. A lock that needs to be acquired by a writer process. Only one process can hold a write lock at the same time.

write-ahead log In logging, a log must be written ahead of the actual update made to the data, in case a crash occurs just after an update is made, and before this update could be recorded in the log.

Index

A

ACID properties 9
 atomicity 9
 consistency 9
 durability 9, 39
 isolation 9

atomicity 8
 failure atomicity 9, 39

auto-commit mode 47

C

concurrency control
 comparison of methods for 37
 optimistic algorithm 28
 pessimistic algorithm 28, 32
 time-stamp ordering 32
 transaction 28

concurrent versioning system 56

conflicting operations 19–21, 26

contract law 9

crash 39
 recovery 40
 resilience 39

cycle detection algorithm 25

D

data
 recovery 39
 relation 44, 51
 replication 54–55

data ownership
 master–slave 55
 peer-to-peer (P2P) environment 55
 update anywhere 55
 workflow 55

database 7
 connecting to 45
 consistent state 17
 driver 45
 inconsistent state 8
 metadata 45
 query 44
 record 44
 relational 43
 table 44

database management system (DBMS) 7, 43, 45
 distributed 53, 57–58
 query language 44
 transaction support 46

deadlock
 four conditions 31
 two-phase locking 31

deadlock condition
 circular wait 31
 exclusive access 31
 hold while waiting 31
 no pre-emption 31

directed acyclic graph (DAG) 25

F

failure transparency 39

fragmentation 53, 58
 horizontal slicing 54, 58
 vertical slicing 54, 58

J

Java Transaction API (JTA) 10, 47
 interface 12

JDBC 44–45
 Connection interface 46–47
 driver 46
 resource manager 47

L

lock
 exclusive 31
 granularity of 30
 read 31
 shareable 31
 write 31

logging
 see rollback

lost update problem 18

O

Open Database Connectivity (ODBC) 45

open source projects 56

optimistic concurrency control 28, 37
 update phase 37
 validation phase 37
 working phase 37

P

- persistence 40
- persistent storage 7, 40, 42
- precedence graph 20, 24
 - cycle 23

R

- read
 - dirty 50
 - non-repeatable 50
 - phantom 50
- redundant array of independent disks (RAID) 40
- replication
 - asymmetric 55
 - asynchronous 55
 - synchronous 55
- rollback 41
 - logging 41
 - rollback method 47
 - shadowing 41
 - strategy 39
- rollback log 41
 - after image 41
 - before image 41
 - write-ahead 41

S

- serialisability 15, 32
 - definition 18
 - of a schedule 21, 25, 28
 - testing 20
- shadowing
 - see rollback
- stable storage 40, 42
- Structured Query Language (SQL) 44
 - library 47
 - ResultSet 49
 - select 49

T

- time-stamp ordering (TSO) 28, 32, 37
 - and 2PL 36
 - cascade of aborts 35
 - log 34
- topological sort 24

transaction

- abort 9, 40
- all or nothing 40
- atomic 40
- coarse-grained action 8
- commit 9, 40
- commit protocol (for distributed transactions) 56
- concurrency control 28
- distributed 47, 53
- externally visible effect 40
- global 56
- ID 11
- interleaving 16, 18
- intermediate state 18
- isolation 36
- isolation level 49–50
- local 56
- schedule 15
- serial schedule 15, 17
- timeout 31–32
- validation 37

transaction processing (TP) system 13, 18

- cache manager 13
- recovery manager 13
- resource manager 47
- scheduler 13, 19, 25
- transaction manager 13

transaction processing monitor

- see transaction processing (TP) system

transaction state

- Aborted 11
- Active 11
- Committed 11
- Failed 11
- Partially Committed 11

transparent management (of a database) 54

two-phase commit (2PC) protocol 56

- coordinator 56
- prepare phase 56
- resolution phase 56

two-phase locking (2PL) 28, 30, 37

- acquire phase 30
- deadlock 31
- release phase 30
- strict 30

V

- volatile memory 7, 42

